

INFORMATIQUE • 3ÈME

INFORMATIQUE – CLASSE DE 3ÈME

LEÇON N04

ORGANISATION DES DONNÉES
EN INFORMATIQUE

DURÉE CONSEILLÉE : 3 SÉANCES (FICHE DE PROGRESSION HARMONISÉE)

DANS CETTE LEÇON

Cette leçon explique comment les données sont organisées de manière structurée (fichiers, dossiers, tableaux) et comment elles sont représentées et codifiées dans l'ordinateur grâce au système binaire et aux codes comme l'ASCII. Elle relie ces notions à des situations concrètes de la vie courante camerounaise.

- À la fin de cette leçon, je sais définir une donnée, une information et distinguer les principaux types de données (texte, nombre, image, son).
- À la fin de cette leçon, je sais expliquer ce qu'est une organisation structurée des données (fichiers, dossiers, arborescence, tableau).
- À la fin de cette leçon, je sais organiser des données de la vie courante dans une arborescence de dossiers et de fichiers bien nommés.
- À la fin de cette leçon, je sais décrire le rôle de l'unité centrale, de la mémoire et des supports de stockage dans la représentation des données.
- À la fin de cette leçon, je sais convertir un nombre entier du système décimal vers le binaire et inversement.
- À la fin de cette leçon, je sais expliquer le principe de la codification des caractères (code ASCII) et coder/décoder un message court.

TÉLÉCHARGE
ONBUCHGratuit • Google Play
Tuteur IA, annales,
concours, résultats

PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Données, informations et types de données
2. Organisation structurée des données
3. Représentation des données dans l'ordinateur
4. Codification des nombres : le binaire
5. Codification des caractères et des autres données
6. Synthèse et applications à la vie courante
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir une donnée, une information et distinguer les principaux types de données (texte, nombre, image, son).
- À la fin de cette leçon, je sais expliquer ce qu'est une organisation structurée des données (fichiers, dossiers, arborescence, tableau).
- À la fin de cette leçon, je sais organiser des données de la vie courante dans une arborescence de dossiers et de fichiers bien nommés.
- À la fin de cette leçon, je sais décrire le rôle de l'unité centrale, de la mémoire et des supports de stockage dans la représentation des données.
- À la fin de cette leçon, je sais convertir un nombre entier du système décimal vers le binaire et inversement.
- À la fin de cette leçon, je sais expliquer le principe de la codification des caractères (code ASCII) et coder/décoder un message court.
- À la fin de cette leçon, je sais citer les unités de mesure de la quantité de données (bit, octet, Ko, Mo, Go) et effectuer des conversions simples.
- À la fin de cette leçon, je sais rédiger et justifier une démarche de résolution de problème concret utilisant l'organisation et la codification des données.

PRÉREQUIS

- Notion de fichier et de dossier vue en classe de 4ème

- Manipulation élémentaire du clavier et de la souris
- Notion de système de numération décimale (mathématiques)
- Vocabulaire de base : ordinateur, mémoire, clé USB, disque dur
- Opérations élémentaires sur les puissances de 2 (calcul mental)

1 DONNÉES, INFORMATIONS ET TYPES DE DONNÉES

Au marché Mokolo de Yaoundé, la maman d'Amina tient un cahier où elle note chaque jour ses ventes de tomates : date, quantité, prix. À la fin du mois, elle passe des heures à tout additionner et se trompe souvent. Son neveu, élève en 3ème, lui montre qu'un téléphone ou un ordinateur peut ranger ces mêmes informations de façon ordonnée et faire les calculs en quelques secondes. Mais comment l'ordinateur range-t-il ces données ? Comment transforme-t-il les noms, les nombres et les images en suites de 0 et de 1 ? C'est ce que nous allons découvrir.

1.1 De la donnée brute à l'information : définitions et distinction

Avant de parler d'ordinateur, posons les bases. Dans la vie courante, nous sommes entourés de faits, de chiffres, de mots, d'images. Tant qu'ils sont isolés, sans lien ni traitement, ce ne sont que des **données brutes**. Dès qu'on les organise, qu'on les calcule, qu'on les met en perspective pour prendre une décision, elles deviennent de l'**information**.

DÉFINITION Donnée et information

Une **donnée** (ou **donnée brute**) est un élément élémentaire, non traité, qui représente un fait, une mesure, un symbole ou une observation brute (ex. : le chiffre 18, le mot « Yaoundé », un pixel rouge). Elle n'a pas de sens exploitable par elle-même.

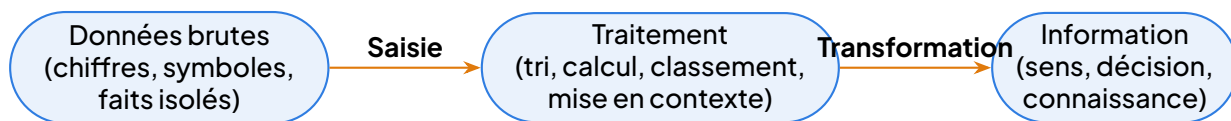
Une **information** est une donnée qui a été traitée, organisée et contextualisée pour avoir un sens précis et utile pour un destinataire (ex. : « La température à Yaoundé est de 18 °C ce matin »). L'information permet de prendre une décision ou de comprendre une situation.

Le passage de la donnée à l'information, c'est le cœur du travail de l'informaticien : le **traitement de l'information**.

EXEMPLE Exemple camerounais : pluviométrie à Douala

Imaginons une station météo à Douala. Chaque jour à 6 h, un capteur relève la hauteur de pluie : 12, 5, 0, 23, 1, 5, 0, 0, 0, 45, 2 (en millimètres).

- Ces sept nombres sont des données brutes. Seuls, ils ne disent pas s'il a beaucoup plu cette semaine.
- Le météorologue les traite : il fait la somme (85,8 mm), la moyenne (12,3 mm/jour), repère le maximum (45,2 mm le dimanche). Il rédige : « Semaine très arrosée, cumuls supérieurs à la normale saisonnière, risque d'inondations dans les bas-fonds. »
- Ce bulletin est une information : la mairie peut décider d'envoyer des pompes, les habitants de ranger leurs affaires.



Chaîne de transformation : de la donnée brute à l'information

Pour bien fixer la distinction, comparons systématiquement données brutes et informations dans un tableau d'exemples locaux :

Situation	Donnée brute (exemple)	Information obtenue
Bulletin scolaire	Notes : 12, 14, 9, 16, 11	« Moyenne 12,4 — Admis au BEPC, mention Assez Bien »
Gestion boutique	Ventes du jour : 5 kg, 2 kg, 8 kg, 3 kg	« Total 18 kg vendus → Chiffre d'affaires 45 000 FCFA, stock restant 12 kg »
Carnet de santé	Températures : 38,2 ; 38,5 ; 37,8 ; 37,0	« Fièvre en baisse, traitement efficace, fin dans 2 jours »
Relevé téléphonique	Appels : 06xx... (12 min), 06yy... (3 min)	« Forfait dépassé de 15 min → Facture majorée de 1200 FCFA »
Photo d'identité	Pixels RVB : (255,200,180), (254,199,179)...	« Photo conforme aux normes CNI : fond clair, visage visible »

ATTENTION Erreur fréquente : confondre donnée brute et information

Beaucoup d'élèves (et d'adultes !) disent : « J'ai une information : le numéro de téléphone de Paul est 6 XX XX XX XX. » Faux. Ce numéro, tant qu'il n'est pas utilisé pour appeler, situer géographiquement ou identifier un abonné, n'est qu'une donnée brute (une suite de chiffres). Il devient information quand on l'exploite : « Je compose le 6 XX XX XX XX pour joindre Paul » ou « Ce numéro commence par 6, c'est un mobile MTN Cameroun ». Astuce : Demandez-vous toujours : « Est-ce que cela me permet d'agir ou de comprendre quelque chose maintenant ? » Si oui, c'est de l'information.

1.2 Les principaux types de données

L'ordinateur ne « comprend » pas le sens des mots ni des images. Il ne manipule que des **nombre**s **binaires** (des 0 et des 1). Pourtant, il gère votre texte, vos photos, vos musiques, vos vidéos. Comment ? En **codifiant** chaque type de donnée selon une convention précise. Au programme de 3ème, nous identifions cinq grands types de données élémentaires.

DÉFINITION Types de données élémentaires

1. **Texte** : suite de caractères (lettres, chiffres, signes de ponctuation). Ex. : un SMS, un mot de passe, un article de journal.
2. **Nombre** : valeurs numériques (entiers, décimaux, négatifs). Ex. : une note sur 20, un prix en FCFA, une température.
3. **Image** : ensemble de points (pixels) codés par leur couleur. Ex. : une photo de classe, un logo, une radiographie.
4. **Son** : signal audio numérisé (échantillons). Ex. : un message vocal WhatsApp, la sonnerie du téléphone, l'hymne national.
5. **Vidéo** : séquence d'images (frames) synchronisée avec une piste son. Ex. : un film Nollywood, un tutoriel YouTube, une visioconférence.

Chaque type a sa propre structure et sa propre codification (que nous verrons en sections 4 et 5).

Reprenons nos exemples de la vie courante pour les classer :

EXEMPLE Données du quotidien classées par type

- Notes d'un élève (12, 14, 9, 16) → Nombre (entiers ou décimaux).
- Numéro de téléphone (6 XX XX XX XX) → Texte (suite de caractères chiffrés, on ne fait pas de calculs dessus).
- Photo de famille → Image (matrice de pixels).
- Message vocal WhatsApp (1 minute) → Son (fichier audio compressé, ex. Opus/OGG).
- Cérémonie de remise des prix filmée → Vidéo (suite d'images + son).

Remarque : Un même objet peut contenir plusieurs types. Une page web = texte + images + vidéos + sons. Un fichier tableur = nombres + texte + formules (qui sont du texte exécutable).

EXERCICE RÉSOLU 1 Calcul de la moyenne d'une classe : du brut à l'info

Énoncé. Un professeur de 3ème a relevé les notes sur 20 de 6 élèves à un devoir d'informatique : 14 ; 9 ; 17 ; 11 ; 13 ; 16. Il veut obtenir l'information « Moyenne de la classe » pour le bulletin. Données brutes (entrée) : Liste des 6 nombres. Traitement (algorithme) :

```
Algorithme CalculMoyenne
Variables
  notes : Tableau[1..6] d'Entier
```

```
somme, effectif : Entier
moyenne : Reel
Debut
notes <- [14, 9, 17, 11, 13, 16]
somme <- 0
Pour i de 1 a 6 faire
    somme <- somme + notes[i]
FinPour
effectif <- 6
moyenne <- somme / effectif
Afficher "Moyenne de la classe : ", moyenne
Fin
```

Information (sortie) : « La moyenne de la classe est de 13,33/20. » Interprétation : Niveau global satisfaisant, pas de redoublement prévisible.

SOLUTION

Solution détaillée. On identifie clairement les étapes :

- Saisie : Les 6 notes sont entrées au clavier ou scannées.
- Stockage : Elles sont placées en mémoire (variables ou tableau).
- Calcul : L'unité arithmétique et logique (UAL) du processeur effectue l'addition et la division.
- Restitution : Le résultat s'affiche à l'écran, s'imprime sur le bulletin, ou s'enregistre dans la base de données de l'établissement.

C'est exactement ce que fait un tableur (LibreOffice Calc, Excel) avec la formule =MOYENNE(A1:A6).

LESAVAIS-TU ? Un message vocal WhatsApp d'une minute représente déjà des centaines de milliers de données binaires

Un son est une onde analogique continue. Pour l'enregistrer, l'ordinateur l'échantillonne : il mesure l'amplitude de l'onde plusieurs milliers de fois par seconde.

- Qualité « vocale » (bande étroite) : 8 000 échantillons/seconde (8 kHz).
- Chaque échantillon est codé sur 16 bits (2 octets).
- Pour 1 minute (60 s) : $8\,000 \times 16 \times 60 = 7\,680\,000$ bits ≈ 960 kOctets (presque 1 Mo) sans compression.

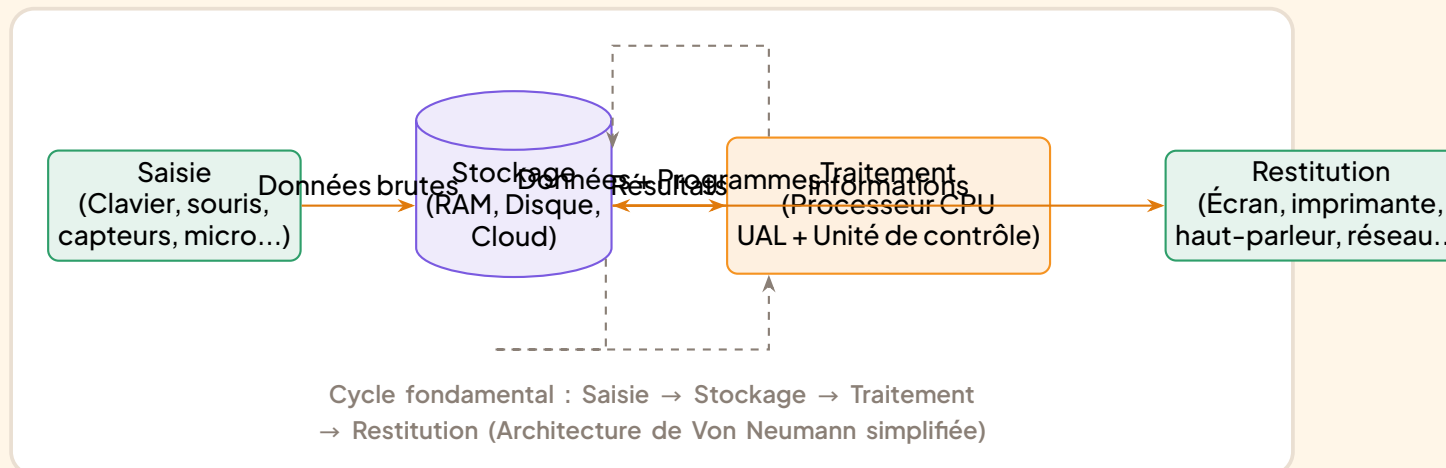
WhatsApp utilise le codec Opus qui compresse fortement (environ 12–24 kb/s). Votre message d'1 minute pèse donc 90 à 180 kOctets seulement. Mais à la base, ce sont bien des millions de 0 et de 1 qui voyagent sur la fibre CAMTEL ou la 4G MTN/Orange pour arriver chez votre correspondant !

1.3 Le cycle de traitement de l'information

Tout système informatique (ordinateur, smartphone, tablette, serveur) suit le même cycle fondamental, hérité de l'architecture de Von Neumann. Comprendre ce cycle, c'est comprendre comment la maman d'Amina au marché Mokolo peut remplacer son cahier par une application simple.

MÉTHODE Les quatre étapes du traitement de l'information

1. **Saisie (Entrée)** : Capture des données brutes via des périphériques d'entrée (clavier, souris, scanner, micro, capteur, caméra, écran tactile). Ex. : Taper les ventes du jour sur le clavier du téléphone.
2. **Stockage (Mémoire)** : Conservation des données et des programmes en mémoire vive (RAM) pendant le traitement, et en mémoire morte (disque dur, SSD, carte SD, cloud) pour la durée. Ex. : L'application enregistre chaque ligne dans un fichier local ou sur Google Drive.
3. **Traitement (Unité Centrale / Processeur)** : Exécution des instructions (calculs, tris, recherches, conversions) par le processeur (CPU). C'est l'étape « intelligente ». Ex. : Somme des quantités, multiplication par le prix unitaire, calcul du total mensuel.
4. **Restitution (Sortie)** : Présentation du résultat (information) à l'utilisateur via des périphériques de sortie (écran, imprimante, haut-parleur, port USB, réseau). Ex. : Affichage « Total mois : 45 000 FCFA » + impression du récapitulatif.



Revenons à Amina et sa maman. Grâce à ce cycle :

- **Saisie** : Maman dicte ou tape « 15 kg tomates à 500 FCFA/kg » sur son smartphone.
- **Stockage** : L'application range la ligne dans une base de données locale (SQLite) ou dans le cloud.
- **Traitement** : Le processeur calcule instantanément $15 \times 500 = 7\,500$ FCFA, met à jour le total journalier, le stock, le bénéfice estimé.

- **Restitution** : L'écran affiche un tableau clair, un graphique des ventes de la semaine, et peut envoyer un résumé par WhatsApp à Amina.

Finis les erreurs d'addition, les cahiers perdus, les heures de calcul. L'informatique, c'est d'abord **automatiser le traitement de l'information** pour gagner du temps et de la fiabilité.

EXPÉRIENCE**TP guidé : Observer le cycle sur votre smartphone**

Objectif : Identifier les 4 étapes du cycle sur une action quotidienne. Matériel : Un smartphone (ou ordinateur) avec une application de notes ou de calculatrice. Manipulation :

1. Ouvrez l'application Calculatrice.
2. Tapez $1500 + 2500 + 3200$ (Saisie : clavier tactile).
3. Appuyez sur = (Traitement : le processeur additionne).
4. Observez l'affichage 7200 (Restitution : écran).
5. Fermez l'application, rouvrez-la : le résultat a disparu (Stockage volatile : RAM non persistante).
6. Refaites le calcul, puis faites une capture d'écran (Stockage persistant : mémoire flash / galerie photos).

Observations : Sans appui sur =, pas de traitement. Sans stockage persistant (capture, note), l'information est perdue à la fermeture. Interprétation : Toute application informatique, du jeu vidéo au logiciel de paie de la fonction publique, repose sur ce même cycle. La différence n'est que dans la complexité du traitement et le volume des données.

À RETENIR**Résumé de la section**

- Une donnée est un fait brut, sans sens direct (chiffre, mot, pixel, échantillon son).
- Une information est une donnée traitée, organisée, qui a du sens et aide à décider.
- Le passage de l'une à l'autre se fait par le traitement de l'information (Saisie → Stockage → Traitement → Restitution).
- Il existe 5 types de données de base : Texte, Nombre, Image, Son, Vidéo. Chacun a sa codification propre.
- Piège : Ne pas confondre le contenu brut (donnée) et son exploitation (information).

2 ORGANISATION STRUCTURÉE DES DONNÉES

2.1 Pourquoi structurer les données ?

Dans la section précédente, nous avons distingué la **donnée** brute (un chiffre, un mot, un son) de l'**information** (la donnée mise en contexte et exploitée). Nous avons aussi vu les **types de données** (entiers, réels, booléens, chaînes de caractères). Mais une donnée isolée ne suffit pas : dans la vie réelle, comme dans l'ordinateur, nous manipulons des collections de données. Imagine un cybercafé à Yaoundé qui gère 500 élèves : sans rangement, retrouver la fiche d'un seul élève prendrait des heures. C'est le problème fondamental de l'**organisation structurée des données** : comment ranger, nommer et relier les données pour les retrouver vite, éviter les pertes et les doublons ?

À RETENIR Le triptyque de l'organisation

Une bonne organisation des données répond à trois impératifs :

1. Accessibilité : retrouver l'information en quelques clics (pas en cherchant pendant 20 minutes).
2. Intégrité : éviter les pertes (fichier effacé par erreur) et les doublons (deux versions différentes de la même facture).
3. Évolutivité : pouvoir ajouter, supprimer ou modifier des données sans casser l'ensemble du système.

2.2 Notion de fichier et de dossier

DÉFINITION Fichier

Un **fichier** est l'unité de base du stockage de l'information sur un support numérique (disque dur, clé USB, SSD). Il est identifié par :

- un nom (ex. bulletin_3eA),
- une extension (suffixe après le point, ex. .txt, .jpg, .mp3, .pdf) qui indique le format des données et donc le logiciel capable de l'ouvrir,
- une taille (exprimée en octets : Ko, Mo, Go),
- une date de création/modification.

LESAVAIS-TU ? L'extension, passeport du fichier

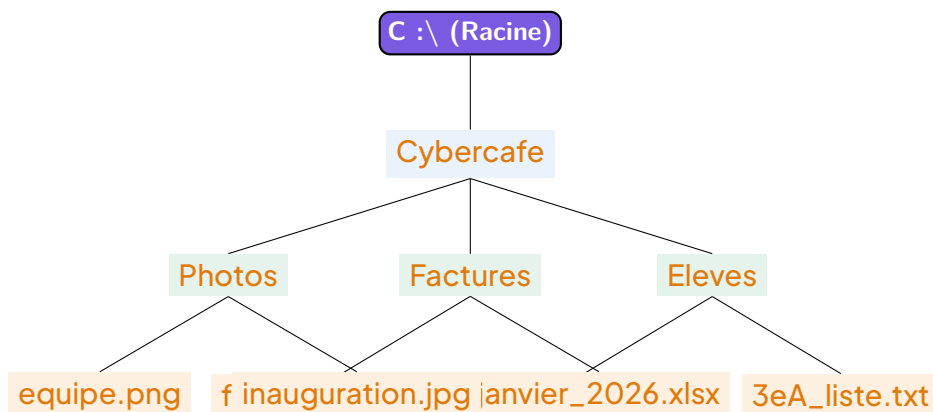
L'extension n'est pas obligatoire pour que le fichier existe, mais elle est indispensable pour l'utilisateur et le système d'exploitation. Sous Windows, masquer les extensions est une source d'erreurs fréquentes (ex. `photo.jpg.exe` qui passe pour une image alors que c'est un programme potentiellement dangereux). Astuce : dans l'Explorateur de fichiers, onglet Affichage, coche Extensions de noms de fichiers.

DÉFINITION Dossier (ou Répertoire)

Un **dossier** (ou **répertoire**) est un conteneur logique qui regroupe des fichiers et/ou d'autres dossiers (appelés **sous-dossiers**). Il ne contient pas de données « métier » (texte, image, son), mais seulement des références vers d'autres objets. Un dossier a un nom, pas d'extension standard, et une taille qui correspond à la somme des tailles de son contenu.

2.3 L'arborescence et le chemin d'accès

Pour organiser les dossiers entre eux, les systèmes de fichiers (NTFS, ext4, FAT32...) utilisent une structure en **arbre inversé** : la **racine** en haut, les branches (dossiers) et les feuilles (fichiers) en bas. C'est l'**arborescence**.



DÉFINITION Chemin d'accès (Path)

Le **chemin d'accès** (ou **path**) est l'adresse unique d'un fichier ou dossier dans l'arborescence. Il se lit de la racine vers la cible, en séparant les noms par un séparateur :

- \\ (antislash) sous Windows (ex. `C:\\Ecole\\3eA\\notes.txt`),

- / (slash) sous Linux (ex. `/home/eleve/notes.txt`).

On distingue le chemin absolu (complet depuis la racine) du chemin relatif (depuis le dossier courant).

EXEMPLE Lecture d'un chemin absolu Windows

Prenons le chemin `C:\Ecole\3eA\Devoirs\DM01_algo.txt`.

1. `C:\` : racine du disque système (lecteur C).
2. `Ecole` : dossier principal à la racine.
3. `3eA` : sous-dossier dans `Ecole`.
4. `Devoirs` : sous-dossier dans `3eA`.
5. `DM01_algo.txt` : fichier cible (extension `.txt` → fichier texte).

Si tu es dans le dossier `3eA`, le chemin relatif vers le même fichier est simplement `Devoirs\DM01_algo.txt`.

2.4 Règles de bon nommage

Un nommage rigoureux évite les erreurs, les problèmes de compatibilité (Linux/Windows) et les pertes de temps.

PROPRIÉTÉ Règles d'or pour nommer fichiers et dossiers

1. Pas d'espaces : utilise le `_` (tiret bas ou underscore) ou le `-` (tiret). Ex. : `liste_eleves_3eA.txt` (bon) vs `liste eleves 3eA.txt` (mauvais).
2. Caractères autorisés : lettres (sans accents de préférence), chiffres, `_`, `-`. Interdits sous Windows : `\ / : * ? " < > |` (réservés au système).
3. Noms explicites et datés : `facture_2026-01-15_boutique_ngono.pdf` est meilleur que `doc1.pdf`.
4. Extension cohérente : ne renomme pas `photo.jpg` en `photo.txt` pour « l'ouvrir avec le Bloc-notes » : cela rend le fichier illisible par les logiciels.
5. Longueur raisonnable : vise moins de 50 caractères pour la lisibilité.

ATTENTION Le piège du « Bureau » et de la racine

Erreur classique : enregistrer tous ses fichiers directement sur le Bureau (Desktop) ou à la racine C:\.

- Conséquences : Bureau encombré → démarrage de l'ordinateur ralenti ; risque de suppression accidentelle ; impossibilité de retrouver un fichier parmi 300 icônes.
- Bonne pratique : crée un dossier `MesTravaux_3e` dans Documents, puis une arborescence par matière/projet avant de créer le premier fichier.

2.5 Méthode : construire une arborescence claire en 4 étapes

MÉTHODE Créer son arborescence de travail

1. Analyser les besoins : quels types de documents ? (Cours, Devoirs, Photos, Projets, Admin). Qui y accède ? (Moi seul, un groupe, le professeur).
2. Dessiner l'arbre sur papier : racine → 3 à 5 dossiers principaux maximum → sous-dossiers (maximum 2 à 3 niveaux de profondeur pour rester simple).
3. Appliquer le nommage : noms courts, sans espaces, sans accents, en minuscules (ex. `cours_maths`, `projet_site_web`).
4. Créer l'arborescence avant le premier fichier : clic droit → Nouveau → Dossier. Puis enregistrer directement dans le bon dossier (Fichier → Enregistrer sous → naviguer vers le dossier cible).

2.6 Organisation en tableau : lignes et colonnes

Au-delà des fichiers, les données à l'intérieur d'un fichier (tableur, fichier CSV) suivent souvent une structure **tabulaire**. C'est la modélisation la plus intuitive : un **tableau** organise les données en lignes et en colonnes.

DÉFINITION Enregistrement et Champ

Dans un tableau structuré :

- Une ligne représente un **enregistrement** : une occurrence complète (ex. une vente, un élève, un produit).
- Une colonne représente un **champ** : une propriété unique (ex. Date, Quantité, Prix). Chaque champ a un nom unique et un type de données (entier, réel, texte, date).

La première ligne contient généralement les en-têtes (noms des champs).

Date	Quantité (kg)	Prix unitaire (FCFA/kg)	Montant (FCFA)	Client
15/01/2026	10	500	5 000	Mme Ngo
16/01/2026	5	550	2 750	M. Biya
17/01/2026	20	500	10 000	Restaurant Le Baobab
18/01/2026	3	600	1 800	Étudiant UY1

Tableau structuré « Ventes de tomates » : 4 enregistrements (lignes), 5 champs (colonnes). Le champ Montant est calculé : Quantité × Prix unitaire.

EXEMPLE Analogie : le cahier de ventes du marché

Mme Ngo, vendeuse de tomates au Marché Central de Douala, tient un cahier. Chaque page = un fichier. Chaque ligne = une vente (enregistrement). Colonnes = Date, Quantité, Prix/kg, Total, Client.

- Recherche : « Combien de kg vendus le 16/01? » → parcourir les lignes en filtrant sur la colonne Date.
- Calcul : « Recette totale? » → somme de la colonne Montant : 5 000 + 2 750 + 10 000 + 1 800 = 19 550 FCFA.
- Tri : « Meilleurs clients? » → tri décroissant sur la colonne Montant.

C'est exactement ce que fait un tableur (LibreOffice Calc, Excel) : structurer pour interroger.

2.7 Analogie avec le classement d'une bibliothèque ou d'un magasin

- **Bibliothèque de l'école** : rayons (dossiers principaux : Romans, Sciences, Histoire) → étagères (sous-dossiers : Physique, Chimie) → livres (fichiers). Chaque livre a une **cote** (chemin d'accès) : Sciences/Physique/Mecanique_Newton. Sans cote, le livre est « perdu » dans la masse.
- **Magasin (boutique « Tout pour l'École »)** : allées (dossiers) → rayons (sous-dossiers) → produits (fichiers). Le code-barres joue le rôle d'identifiant unique de chaque produit. L'inventaire consiste à interroger l'ensemble des données structurées.

EXPÉRIENCE TP guidé : créer l'arborescence de ton année de 3ème

Objectif : mettre en pratique la méthode des 4 étapes pour organiser tes documents scolaires.

Matériel : poste Windows ou Linux, Explorateur de fichiers.

Durée : 20 min.

Manipulation :

1. Ouvre l'Explorateur de fichiers (touche Win+E sous Windows). Va dans Documents.
2. Crée le dossier racine : 3eme_2025-2026 (respecte le nommage!).
3. À l'intérieur, crée ces 5 dossiers principaux : cours, devoirs, projets, admin, ressources.
4. Dans cours, crée un sous-dossier par matière : maths, physique, svt, info, francais, anglais.
5. Dans devoirs, crée ds_1er_trimestre, ds_2eme_trimestre, devoirs_maison.
6. Test : ouvre ton traitement de texte. Écris « Test arborescence ». Enregistre sous test_arborescence.odt directement dans Documents\3eme_2025-2026\cours\info.
7. Vérifie dans l'Explorateur que le fichier est bien là.

Observations attendues :

- L'arborescence est visible, logique, extensible.
- Le chemin d'accès du fichier de test est : C:\Users\TonNom\Documents\3eme_2025-2026\cours\info\test_arborescence.odt
- Si tu cherches « test » dans la barre de recherche de l'Explorateur (en étant dans 3eme_2025-2026), tu le trouves instantanément.

Interprétation : tu viens de construire une structure de rangement physique (sur le disque) qui reflète ta organisation logique (matières, types de documents). C'est la base de toute organisation des données en informatique.

2.8 Synthèse de la section

À RETENIR

Ce qu'il faut retenir de l'organisation structurée

- Fichier : unité de stockage (nom + extension + taille). L'extension indique le programme capable de l'ouvrir.
- Dossier : conteneur logique pour regrouper fichiers et dossiers. Pas d'extension.
- Arborescence : structure en arbre (racine → branches → feuilles). Un chemin d'accès unique par objet.
- Chemin d'accès : adresse absolue (depuis la racine) ou relative (depuis le dossier courant). Séparateur \ (Windows) ou / (Linux).
- Nommage : sans espaces, sans accents, explicite, daté (AAAA-MM-JJ), extension correcte.

- Tableau : lignes = enregistrements (objets), colonnes = champs (propriétés typées). Base du tableur.
- Bonne pratique : créer l'arborescence avant de produire des fichiers. Ne jamais tout mettre sur le Bureau ou à la racine.

EXERCICE RÉSOLU 2 Analyse d'un chemin d'accès et correction de nommage

Énoncé : un élève a enregistré son devoir d'algorithmique sous le nom `Mon Devoir Algo Final V2.docx` sur son Bureau. Le chemin affiché est `C:\Users\Kamel\Desktop\Mon Devoir Algo Final V2.docx`.

1. Identifie la racine, les dossiers, le fichier et l'extension.
2. Cite 3 erreurs de nommage dans le nom du fichier.
3. Propose un nom correct selon les règles vues en cours.
4. Où aurait-il dû l'enregistrer idéalement (propose une arborescence cohérente)?

SOLUTION

Solution détaillée :

1. Racine : `C:\`. Dossiers : `Users`, `Kamel`, `Desktop`. Fichier : `Mon Devoir Algo Final V2`. Extension : `.docx` (document Word ou LibreOffice Writer).
2. Erreurs : (1) espaces multiples (posent problème dans les commandes et certains logiciels); (2) majuscules incohérentes (pas de convention); (3) « V2 » non standard (mieux vaut une date ou un numéro de version propre); (4) « Final » trompeur (souvent ce n'est pas le dernier!).
3. Nom correct : `devoir_algo_2026-01-20_v02.docx`. Format de date clair (AAAA-MM-JJ), tirets bas, minuscules, version numérotée.
4. Arborescence idéale : `C:\Users\Kamel\Documents\3eme_2025-2026\devoirs\devoirs_maison\devoir_`

EXERCICE 1 Entraînement personnel

1. Dessine l'arborescence (sous forme d'arbre) de ton téléphone portable : racine `Stockage_interne` → dossiers `DCIM`, `Download`, `WhatsApp`, `Documents` → au moins 2 fichiers réalistes par dossier (ex. `IMG_20260115_1030.jpg`, `cv_kamel.pdf`). Indique les extensions.
2. Donne le chemin d'accès absolu (style Windows) du fichier `cv_kamel.pdf` dans ton arborescence.

3. Tu reçois un fichier nommé `facture_mars_2026.xlsx`. Tu veux le ranger. Décris précisément (clics et dossiers) où tu le places dans ton arborescence `3eme_2025-2026` créée dans le TP, et pourquoi.
4. Un camarade t'envoie un fichier nommé `photo_vacances` (sans extension). Tu double-cliques : Windows demande « Comment voulez-vous ouvrir ce fichier ? ». Explique pourquoi et comment résoudre le problème sans installer de logiciel.

CORRIGÉ Exercice — Organisation structurée

1. **Arbre attendu :** `racine` `Stockage_interne` → `DCIM` (`IMG_20260115_1030.jpg`, `VID_20260114_1400.mp4`), `Download` (`cv_kamel.pdf`, `cours_algo.pdf`), `WhatsApp` (`IMG-20260110-WA0001.jpg`, `IMG-20260111-WA0002.jpg`), `Documents` (`lettre_motivation.odt`, `notes_physique.txt`).
2. `Stockage_interne\Download\cv_kamel.pdf` (style Windows). Sous Android, on écrirait plutôt `/storage/emulated/0/Download/cv_kamel.pdf` (style Linux).
3. On le place dans `Documents\3eme_2025-2026\admin\facture_mars_2026.xlsx`. Le dossier `admin` regroupe les documents officiels (factures, inscriptions). Le nom du fichier est déjà conforme aux règles de nommage (pas d'espaces, date claire, extension correcte), il n'y a donc rien à renommer.
4. L'extension manque → le système ne connaît pas le format du fichier ni le programme associé. Solution : clic droit → Renommer → ajouter l'extension correcte, par exemple `photo_vacances.jpg` (si c'est une photo). Puis double-cliquer : le fichier s'ouvre avec la visionneuse d'images.

3 REPRÉSENTATION DES DONNÉES DANS L'ORDINATEUR

Dans la section précédente, nous avons appris à organiser les données de manière structurée : un tableau pour les élèves d'une classe, un formulaire pour les clients d'une boutique. Mais une question fondamentale reste en suspens : une fois bien organisées, ces données, comment l'ordinateur les représente-t-il physiquement ? Comment une machine faite de circuits électriques peut-elle mémoriser le prénom « Amina », la note 15/20 ou une photo de famille ? C'est ce que nous allons découvrir maintenant, en partant du plus petit élément possible jusqu'aux clés USB et disques durs que tu utilises chaque jour.

3.1 L'ordinateur ne comprend que deux états

Une machine électrique

Ouvre (en imagination !) le boîtier d'un ordinateur : tu y trouverais des millions de minuscules

composants électroniques appelés **transistors**. Un transistor fonctionne comme un interrupteur : soit le courant électrique **pass**e, soit il **ne passe pas**. Il n'existe pas de troisième possibilité.

C'est une contrainte physique majeure : l'ordinateur ne peut distinguer que **deux états** :

- **absence de courant** (interrupteur ouvert, tension nulle) ;
- **présence de courant** (interrupteur fermé, tension présente).

Toute l'ingéniosité de l'informatique consiste donc à représenter toutes les données — nombres, textes, images, sons, vidéos — uniquement avec ces deux états. On leur associe par convention les symboles **0** (absence de courant) et **1** (présence de courant).

À RETENIR L'idée fondamentale

Un ordinateur est une machine électrique qui ne connaît que deux états : courant présent ou courant absent. Toute donnée, quelle qu'elle soit, est donc codée à l'aide de deux symboles seulement : 0 et 1. C'est ce qu'on appelle le codage **binaire**.

LESAVAIS-TU ? Pourquoi deux états et pas dix ?

Nous comptons en base 10 parce que nous avons dix doigts. Pour une machine, fabriquer un circuit qui distingue dix niveaux de tension différents serait fragile : la moindre perturbation électrique provoquerait des erreurs. Avec seulement deux états très éloignés (« rien » ou « plein courant »), la machine est rapide et très fiable. C'est le choix fait dès les premiers ordinateurs, dans les années 1940, et il n'a jamais changé depuis.

3.2 Le bit et l'octet : les briques de base

Le bit : la plus petite unité de donnée

DÉFINITION Bit

Un **bit** (contraction de l'anglais binary digit, « chiffre binaire ») est la plus petite unité de donnée qu'un ordinateur puisse manipuler. Un bit ne peut prendre que deux valeurs : 0 ou 1.

Un bit seul ne dit pas grand-chose : c'est comme une lampe qui ne peut être qu'allumée ou éteinte. Mais en combinant plusieurs bits, on peut représenter des choses de plus en plus riches :

- avec 1 bit : 2 possibilités (0 ou 1) ;
- avec 2 bits : 4 possibilités (00, 01, 10, 11) ;
- avec 8 bits : $2^8 = 256$ possibilités !

L'octet : un groupe de 8 bits

DÉFINITION Octet

Un **octet** (en anglais byte) est un groupe de 8 bits. C'est l'unité de base utilisée pour mesurer la quantité de données. Un octet peut prendre $2^8 = 256$ valeurs différentes (de 00000000 à 11111111).

Pourquoi 8 bits ? Parce que c'est justement ce qu'il faut pour coder **un caractère** : une lettre, un chiffre, un signe de ponctuation. Avec 256 combinaisons possibles, on a largement de quoi représenter les 26 lettres en majuscules et minuscules, les 10 chiffres, les accents (é, è, à...) et les symboles.

EXEMPLE Un caractère ≈ 1 octet

Le mot « CAMEROUN » contient 8 caractères. Il occupe donc environ 8 octets en mémoire. La phrase « J'aime l'informatique ! » contient 24 caractères (espaces et apostrophe compris), soit environ 24 octets.

ATTENTION Erreur fréquente : confondre bit et octet

Le bit et l'octet ne sont pas la même chose ! Retiens bien :

$$1 \text{ octet} = 8 \text{ bits}$$

Piège classique : un débit de connexion de 8 mégabits par seconde ne permet pas de télécharger 8 mégaoctets par seconde, mais seulement 1 mégaoctet par seconde (car $8 \div 8 = 1$). Astuce : en abrégé, le bit s'écrit avec un « b » minuscule (Mb) et l'octet avec un « o » (Mo).

3.3 Les unités de mesure : Ko, Mo, Go, To

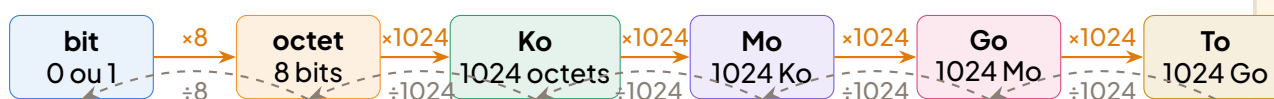
Un octet, c'est très petit : un seul caractère. Une photo en contient des millions ! Pour éviter de manipuler des nombres gigantesques, on a défini des unités plus grandes.

PROPRIÉTÉ Relations entre les unités de stockage (admises)

Les unités de mesure des données sont liées par des multiplications par 1024 :

$$\begin{aligned}1 \text{ Ko (kilo-octet)} &= 1024 \text{ octets} \\1 \text{ Mo (méga-octet)} &= 1024 \text{ Ko} \\1 \text{ Go (giga-octet)} &= 1024 \text{ Mo} \\1 \text{ To (téra-octet)} &= 1024 \text{ Go}\end{aligned}$$

Pourquoi 1024 et pas 1000 ? Parce que l'ordinateur travaille en binaire, et $1024 = 2^{10}$ est la puissance de 2 la plus proche de 1000. C'est un choix pratique des informaticiens.



Hiérarchie des unités de données : on multiplie pour monter vers les grandes unités, on divise pour redescendre.

3.4 Convertir entre les unités

MÉTHODE Convertir des unités de stockage

Pour convertir une quantité de données d'une unité à une autre :

1. Repère le sens de la conversion sur le schéma de la hiérarchie (bit → octet → Ko → Mo → Go → To).
2. Si tu passes d'une grande unité vers une petite unité (ex. Go → Mo), tu multiplies par 1024 à chaque étape.
3. Si tu passes d'une petite unité vers une grande unité (ex. Mo → Go), tu divises par 1024 à chaque étape.
4. S'il y a plusieurs étapes (ex. Go → Ko), multiplie ou divise autant de fois qu'il y a d'étapes.

EXERCICE RÉSOLU 3 Conversions d'unités de stockage

Énoncé. (a) La clé USB de Manga a une capacité de 2 Go. Exprime cette capacité en Mo. (b) Un dossier de photos occupe 5120 Mo. Exprime cette taille en Go. (c) Combien d'octets y a-t-il dans 3 Ko ?

SOLUTION

Solution détaillée.

(a) On passe d'une grande unité (Go) vers une petite unité (Mo) : on multiplie par 1024.

$$2 \text{ Go} = 2 \times 1024 \text{ Mo} = 2048 \text{ Mo}$$

(b) On passe d'une petite unité (Mo) vers une grande unité (Go) : on divise par 1024.

$$5120 \text{ Mo} = \frac{5120}{1024} \text{ Go} = 5 \text{ Go}$$

(c) On passe des Ko aux octets : on multiplie par 1024.

$$3 \text{ Ko} = 3 \times 1024 \text{ octets} = 3072 \text{ octets}$$

Vérification mentale : quand on descend vers une petite unité, le nombre doit augmenter ; quand on monte vers une grande unité, il doit diminuer. Si ton résultat va dans le mauvais sens, tu as inversé multiplication et division !

3.5 Où sont stockées les données ?

Les données codées en binaire doivent être conservées quelque part. L'ordinateur utilise deux grandes familles de mémoires, aux rôles complémentaires.

La mémoire vive (RAM) : la mémoire de travail

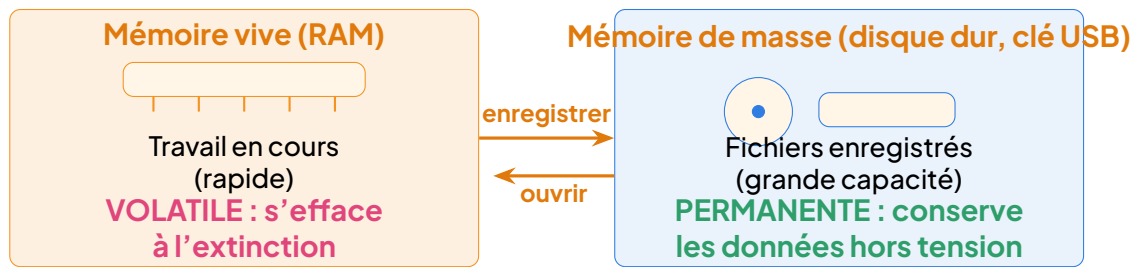
La **mémoire vive** ou **RAM** (Random Access Memory) est la mémoire dans laquelle l'ordinateur place les programmes et les données **pendant qu'il travaille**. Quand tu rédiges une lettre dans un traitement de texte, le texte en cours de saisie se trouve dans la RAM.

Sa caractéristique essentielle : elle est **volatile**, c'est-à-dire **temporaire**. Dès qu'on éteint l'ordinateur (ou qu'il y a une coupure de courant — fréquentes chez nous !), tout ce qui se trouve dans la RAM est **perdu**. C'est pour cela qu'il faut enregistrer régulièrement son travail.

Les mémoires de masse : le stockage permanent

Les **mémoires de masse** conservent les données de façon **permanente**, même lorsque l'appareil est éteint. On y trouve :

- le **disque dur** de l'ordinateur (souvent 500 Go ou 1 To) ;
- la **clé USB** (4 Go, 8 Go, 32 Go...) ;
- la **carte mémoire** des téléphones et appareils photo ;
- le disque dur externe.



La RAM (volatile) contient le travail en cours; la mémoire de masse (permanente) conserve les fichiers enregistrés.

À RETENIR RAM ou mémoire de masse ?

Critère	RAM	Mémoire de masse
Durée de conservation	Temporaire (volatile)	Permanente
Rôle	Travail en cours	Stockage des fichiers
Exemples	Barrette mémoire du PC	Disque dur, clé USB, carte mémoire
Capacité courante	4 à 8 Go	32 Go à 1 To et plus

ATTENTION Coupure de courant = travail perdu

Tu rédiges depuis une heure un exposé au cybercafé et le courant se coupe : si tu n'as pas enregistré ton fichier sur le disque dur ou une clé USB, tout ce qui était dans la RAM est perdu. Prends l'habitude d'enregistrer ton travail toutes les quelques minutes (raccourci Ctrl + S)!

3.6 Ordres de grandeur et applications à la vie courante

Pour te donner une intuition concrète des tailles de fichiers, voici quelques valeurs typiques :

Type de donnée	Taille approximative
Une page de texte	quelques Ko
Une photo prise avec un téléphone	environ 4 Mo
Une chanson (fichier audio)	environ 4 à 5 Mo
Un film	environ 1 à 2 Go (700 Mo à 2 Go)
Un système d'exploitation complet	plusieurs Go

EXEMPLE Combien de photos sur une clé USB de 8 Go ?

Une clé USB de 8 Go peut-elle contenir les 2000 photos du mariage de ta cousine, chaque photo occupant environ 4 Mo ?

Étape 1 : convertir la capacité de la clé en Mo (grande unité → petite unité : on multiplie).

$$8 \text{ Go} = 8 \times 1024 = 8192 \text{ Mo}$$

Étape 2 : diviser la capacité par la taille d'une photo.

$$\frac{8192 \text{ Mo}}{4 \text{ Mo/photo}} = 2048 \text{ photos}$$

Conclusion : la clé peut contenir environ 2000 photos de 4 Mo (2048 exactement, en négligeant l'espace pris par le système de fichiers). Le mariage est sauvé !

EXPÉRIENCE Observer les tailles de fichiers et la mémoire d'un ordinateur

Objectif : vérifier concrètement les notions d'octet, de Ko/Mo/Go et distinguer RAM et stockage.

Matériel : un ordinateur de la salle informatique (Windows), une clé USB contenant quelques fichiers (photos, musique, documents).

Manipulation :

1. Insère la clé USB, ouvre l'explorateur de fichiers et fais un clic droit sur la clé → Propriétés. Note sa capacité totale et l'espace disponible (en Go).
2. Affiche le contenu de la clé en mode Détails : la colonne Taille indique la taille de chaque fichier en Ko ou Mo. Compare la taille d'un document texte et d'une photo.
3. Fais un clic droit sur Ce PC → Propriétés : repère la quantité de Mémoire installée (RAM) et compare-la à la capacité du disque dur.

Observations attendues : les fichiers texte sont minuscules (quelques Ko), les photos bien plus grosses (quelques Mo) ; la RAM (ex. 4 Go) est beaucoup plus petite que le disque dur (ex. 500 Go).

Interprétation : la RAM, petite et rapide, sert au travail momentané ; le disque dur, grand et permanent, sert à conserver tous les fichiers. Les tailles affichées confirment les relations 1 Ko = 1024 octets, 1 Mo = 1024 Ko, etc.

EXERCICE 2 À toi de jouer

1. Convertis : (a) 4 Go en Mo ; (b) 3072 Ko en Mo ; (c) 5 Ko en octets ; (d) 16 bits en octets.
2. La carte mémoire de l'appareil photo du club informatique a une capacité de 16 Go.

Une photo occupe environ 4 Mo. Combien de photos la carte peut-elle contenir?

3. Une clé USB de 4 Go contient déjà 3 films de 1 Go chacun. Peut-on encore y copier 200 chansons de 5 Mo? Justifie ta réponse par un calcul.

CORRIGÉ Exercice « À toi de jouer »

1. (a) $4 \times 1024 = 4096$ Mo. (b) $3072 \div 1024 = 3$ Mo. (c) $5 \times 1024 = 5120$ octets. (d) $16 \div 8 = 2$ octets.
2. $16 \text{ Go} = 16 \times 1024 = 16384$ Mo; $16384 \div 4 = 4096$ photos.
3. Espace occupé : $3 \times 1 = 3$ Go, donc espace libre : $4 - 3 = 1$ Go = 1024 Mo. Les 200 chansons occupent $200 \times 5 = 1000$ Mo. Comme $1000 < 1024$, oui, on peut les copier (il restera environ 24 Mo).

Nous savons désormais que toute donnée est une suite de 0 et de 1, mesurée en octets, Ko, Mo, Go ou To, et conservée soit temporairement dans la RAM, soit durablement sur une mémoire de masse. Reste à comprendre comment ces 0 et ces 1 représentent concrètement un nombre : c'est l'objet de la prochaine section, consacrée au codage binaire des nombres.

4

CODIFICATION DES NOMBRES : LE BINAIRE

Dans la section précédente, nous avons vu que l'ordinateur ne manipule que des **signaux électriques** à l'intérieur de ses circuits. Pour représenter des nombres, du texte, des images ou des sons, il faut trouver une convention d'écriture qui ne repose que sur **deux états distincts**. C'est exactement le rôle du **système binaire** : c'est le « langage maternel » de la machine, celui que le processeur, la mémoire et les bus comprennent nativement. Avant d'apprendre à convertir des nombres, comprenons pourquoi ce choix s'impose physiquement.

4.1 Pourquoi le binaire ? Deux états électriques seulement

Un transistor, composant de base des puces électroniques, fonctionne comme un interrupteur microscopique : il laisse passer le courant (**état passant**) ou il le bloque (**état bloqué**). On associe conventionnellement :

- l'état **passant** (présence de tension, ex. +5 V ou +3,3 V) au chiffre 1;
- l'état **bloqué** (absence de tension, 0 V) au chiffre 0.

DÉFINITION Système binaire (base 2)

Le **système binaire** est un système de numération de position qui utilise seulement deux

symboles : 0 et 1. Chaque chiffre s'appelle un **bit** (contraction de binary digit). La valeur d'un nombre binaire dépend de la position de chaque bit, exactement comme en base 10, mais les puissances sont des puissances de 2 au lieu de puissances de 10.

LESAVAIS-TU ? Le bit, unité fondamentale

Un bit est la plus petite unité d'information en informatique. Un groupe de 8 bits forme un octet (ou byte), qui permet de coder 256 valeurs différentes ($2^8 = 256$). C'est l'unité de base pour mesurer la taille des fichiers (Ko, Mo, Go) et la capacité de la mémoire vive (RAM).

4.2 Rappel : la numération de position en base 10 (décimale)

Avant d'aborder la base 2, rappelons le mécanisme que nous utilisons tous les jours. En base 10, nous avons 10 symboles (0 à 9). La valeur d'un chiffre dépend de sa position : unités, dizaines, centaines, milliers, etc. Ces positions correspondent aux puissances de 10 :

$$\dots 10^3 \quad 10^2 \quad 10^1 \quad 10^0$$

Par exemple, le nombre **2 431** se décompose ainsi :

$$2 \times 10^3 + 4 \times 10^2 + 3 \times 10^1 + 1 \times 10^0 = 2000 + 400 + 30 + 1 = 2431$$

4.3 La numération de position en base 2 (binaire)

Le principe est **identique**, mais la base change : on utilise les puissances de 2.

$$\dots 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

Soit les valeurs : ... 32 16 8 4 2 1.

Poids (Puissance	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bits du nomb	0	0	1	1	0	1
Produit (Bit × P	$0 \times 32 = 0$	$0 \times 16 = 0$	$1 \times 8 = 8$	$1 \times 4 = 4$	$0 \times 2 = 0$	$1 \times 1 = 1$
Somme (Valeur déc					$8 + 4 + 0 + 1 = 13$	

Exemple : $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13_{10}$

Décomposition du nombre binaire 1101 (écrit sur 6 bits : 001101) en somme de puissances de 2. Seuls les bits à 1 contribuent à la somme.

4.4 Conversion binaire → décimal : la somme pondérée

La méthode consiste à identifier les positions des bits à 1, à écrire la puissance de 2 correspondante, et à faire la somme.

MÉTHODE Convertir un nombre binaire en décimal

1. Écrire le nombre binaire en alignant le bit de poids fort (le plus à gauche) sur la plus grande puissance de 2 nécessaire.
2. Pour chaque bit égal à 1, noter la valeur de sa puissance de 2 (2^{position} , position commençant à 0 à droite).
3. Additionner toutes ces valeurs. Le résultat est l'équivalent décimal.

EXEMPLE Conversion de 10110_2 en décimal

Bits :	1	0	1	1	0
Puissances :	2^4	2^3	2^2	2^1	2^0
Valeurs :	16	8	4	2	1
Somme :	$1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1$				
	$= 16 + 4 + 2 = 22_{10}$				

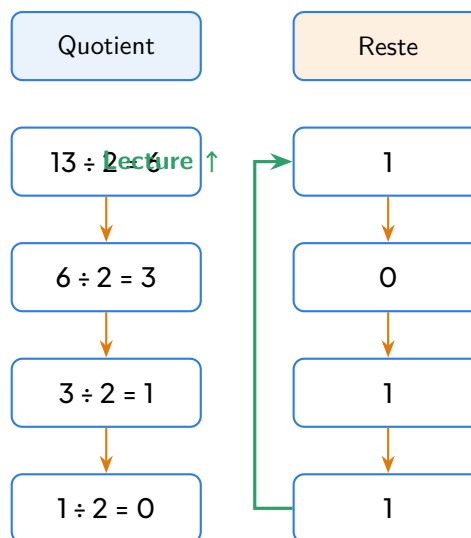
Donc $10110_2 = 22_{10}$.

4.5 Conversion décimal → binaire : divisions successives par 2

C'est l'opération inverse. On divise le nombre décimal par 2 successivement jusqu'à obtenir un quotient nul. Les **restes** (0 ou 1), lus **de bas en haut** (du dernier au premier), forment le nombre binaire.

MÉTHODE Convertir un nombre décimal en binaire (divisions successives)

1. Diviser le nombre décimal par 2. Noter le quotient et le reste (0 ou 1).
2. Recommencer avec le quotient obtenu, tant que le quotient est ≥ 1 .
3. Lorsque le quotient devient 0, arrêter.
4. Lire les restes de bas en haut (du dernier reste obtenu au premier). C'est l'écriture binaire.



Résultat : $13_{10} = 1101_2$ (restes lus de bas en haut : 1, 1, 0, 1)

Schéma en escalier des divisions successives de 13 par 2.
La lecture des restes se fait de bas en haut (flèche verte).

ATTENTION Piège classique : le sens de lecture des restes

C'est l'erreur numéro 1 chez les débutants : lire les restes dans l'ordre où on les a trouvés (de haut en bas).

- Mauvais : $13 \rightarrow$ restes 1, 0, 1, 1 $\rightarrow$ binaire 1011_2 (incorrect, cela vaut 11).
- Bon : On lit de bas en haut : le dernier reste (celui de la division $1 \div 2$) est le bit de poids fort (le plus à gauche). Donc 1, 1, 0, 1 $\rightarrow 1101_2$.

Astuce : Le premier reste obtenu est toujours le bit de poids faible (le plus à droite, 2^0). Le dernier reste est le bit de poids fort.

4.6 Table de correspondance : nombres de 0 à 15

Il est utile de mémoriser les 16 premières valeurs (4 bits) pour gagner du temps lors des exercices ou de la lecture de dumps mémoire.

Décimal	Binaire (4 bits)	Décimal	Binaire (4 bits)
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

À RETENIR Propriété : nombre de valeurs codables

Avec n bits, on peut coder exactement 2^n valeurs différentes (de 0 à $2^n - 1$).

- 4 bits → 16 valeurs (0 ... 15)
- 8 bits (1 octet) → 256 valeurs (0 ... 255)
- 16 bits → 65 536 valeurs

4.7 Exemple résolu complet : le nombre 13 sous toutes ses coutures

EXERCICE RÉSOLU 4 Codage du nombre 13

Énoncé : Montrer par les deux méthodes que le nombre décimal 13 s'écrit 1101 en binaire.

Solution détaillée :

Méthode 1 : Puissances de 2 (Binaire → Décimal) — Vérification On cherche la décomposition de 13 en puissances de 2.

- $2^3 = 8 \leq 13$. On pose un 1 au rang 2^3 . Reste : $13 - 8 = 5$.
- $2^2 = 4 \leq 5$. On pose un 1 au rang 2^2 . Reste : $5 - 4 = 1$.
- $2^1 = 2 > 1$. On pose un 0 au rang 2^1 .
- $2^0 = 1 \leq 1$. On pose un 1 au rang 2^0 . Reste : 0.

Bits obtenus (poids forts → poids faibles) : 1 1 0 1. Vérification : $1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 13$.



Méthode 2 : Divisions successives (Décimal → Binaire)

Division	Quotient	Reste
$13 \div 2$	6	1 (LSB – poids faible)
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1 (MSB – poids fort)

Lecture des restes de bas en haut : 1101. Résultat : $13_{10} = 1101_2$.

4.8 Complément : le système hexadécimal (base 16) — hors programme strict, culture informatique

Les informaticiens utilisent souvent l'**hexadécimal** comme raccourci du binaire. La base 16 utilise 16 symboles : 0–9 puis A, B, C, D, E, F (valeurs 10 à 15).

- Un chiffre hexadécimal code **exactement 4 bits** (un quartet ou nibble).
- Conversion immédiate : on regroupe les bits par paquets de 4 (en partant de la droite) et on remplace chaque paquet par son équivalent hexa.

EXEMPLE Binaire ↔ Hexadécimal

$1101\ 0110_2$ se groupe en 1101 et 0110.

- $1101_2 = 13_{10} = D_{16}$
- $0110_2 = 6_{10} = 6_{16}$

Donc $11010110_2 = D6_{16}$. C'est beaucoup plus compact à écrire et à lire pour un humain, tout en restant trivial à convertir vers/depuis le binaire pour la machine.

4.9 Synthèse et applications à la vie courante

- **Adressage IP (IPv4)** : Une adresse comme 192.168.1.1 est stockée sur 32 bits (4 octets). $192 = 11000000_2$, $168 = 10101000_2$, etc. Les masques de sous-réseau (ex. /24 ou 255.255.255.0) sont des opérations sur les bits.
- **Couleurs web (RGB)** : #FF5733 signifie : Rouge= $FF_{16} = 255$, Vert= $57_{16} = 87$, Bleu= $33_{16} = 51$. Chaque composante est codée sur 1 octet (8 bits).
- **Droits fichiers (Linux)** : `chmod 755` → $7 = 111_2$ (lecture+écriture+exécution pour le propriétaire), $5 = 101_2$ (lecture+exécution pour groupe/autres). C'est de la manipulation de bits (masques).
- **Capteurs et IoT** : Un capteur de température envoie une valeur sur 12 bits (0 à 4095). Le microcontrôleur lit les registres binaires bruts et applique une formule de conversion.

EXERCICE 3 Entraînement aux conversions

1. Convertir en décimal : 101_2 , 1111_2 , 10010_2 , 110011_2 .
2. Convertir en binaire (méthode des divisions) : 5_{10} , 18_{10} , 31_{10} , 63_{10} .
3. Compléter le tableau :

Décimal	Binaire (8 bits)	Hexadécimal
42		
	10101010	
		3F

4. Contexte camerounais : Un cybercafé à Yaoundé configure un routeur. L'adresse IP du serveur est 192.168.10.25. Écrire les 4 octets de cette adresse en binaire (8 bits chacun).

CORRIGÉ Exercice n°1

1. $101_2 = 5$; $1111_2 = 15$; $10010_2 = 18$; $110011_2 = 51$.
2. $5_{10} = 101_2$; $18_{10} = 10010_2$; $31_{10} = 11111_2$; $63_{10} = 111111_2$.

3.

Décimal	Binaire (8 bits)	Hexadécimal
42	00101010	2A
170	10101010	AA
63	00111111	3F

4. $192 = 11000000_2$; $168 = 10101000_2$; $10 = 00001010_2$; $25 = 00011001_2$. Adresse complète : 11000000.10101000.00001010.00011001.

EXPÉRIENCE TP guidé : « Calculatrice binaire » en algorithmique

Objectif : Écrire un algorithme qui convertit un entier positif saisi au clavier en sa représentation binaire (sans utiliser de fonction toute faite).

Matériel : Papier, crayon, puis éditeur de code (Python, C, ou pseudo-code).

Manipulation :

1. Variables : N (entier, nombre à convertir), R (chaîne de caractères, résultat), Q (entier, quotient), rest (entier, reste).
2. Algorithme :

```
Algorithme Conversion_Binaire
Variables
  N, Q, rest : Entier
  R : Chaîne
Debut
  Afficher("Entier positif a convertir : ")
  Saisir(N)
  Q <- N
  R <- ""
  Si N = 0 Alors
    R <- "0"
  Sinon
    TantQue Q > 0 Faire
      rest <- Q mod 2 // Reste de la division par 2 (0 ou 1)
      R <- Caractere(rest) + R // Concaténation À GAUCHE (lecture bas->haut)
      Q <- Q div 2 // Division entière
    FinTantQue
  FinSi
  Afficher("Binaire : ", R)
Fin
```

3. Test : Exécuter avec $N = 13$, $N = 0$, $N = 255$. Vérifier que l'affichage correspond aux tables.
4. Observation : Pourquoi concatène-t-on `Caractere(rest) + R` et non `R + Caractere(rest)` ?
5. Interprétation : Cela correspond exactement à la lecture des restes « de bas en haut » : le premier reste calculé (poids faible) doit se retrouver à la droite de la chaîne finale.

5 CODIFICATION DES CARACTÈRES ET DES AUTRES DONNÉES

Dans la section précédente, nous avons appris à coder les **nombre**s en binaire : l'ordinateur ne comprend que des 0 et des 1, et tout nombre entier peut s'écrire comme une somme de puissances de 2. Mais un ordinateur ne manipule pas seulement des nombres. Quand tu tapes ton nom au clavier, quand tu rédiges une lettre dans un traitement de texte ou quand tu envoies un message à un camarade, l'ordinateur traite des **caractères** : des lettres, des chiffres, des signes de ponctuation. Il traite aussi des images, des sons, des vidéos. Comment ces données, qui ne sont pas des nombres, peuvent-elles être représentées par des 0 et des 1 ? C'est toute la question de cette section, et la réponse est d'une élégance remarquable : **on associe à chaque donnée un nombre, puis on code ce nombre en binaire.**

5.1 Le principe : donner un numéro à chaque caractère

Une situation de la vie courante.

Imagine le directeur d'un collège de Yaoundé qui veut préparer la liste des élèves de 3^{ème} sans écrire leurs noms en entier, pour gagner du temps. Il décide d'un code secret : chaque élève reçoit un numéro (Aïcha = 1, Bertrand = 2, Chantal = 3...). Désormais, au lieu d'écrire les noms, il écrit les numéros. Celui qui possède la **table de correspondance** peut retrouver les noms ; les autres n'y comprennent rien.

L'ordinateur fait exactement la même chose avec les caractères. Comme il ne sait manipuler que des nombres (eux-mêmes écrits en binaire), on a décidé, une fois pour toutes, d'associer un **numéro unique à chaque caractère** : la lettre A, la lettre a, le chiffre 0, le point, l'espace, etc. Écrire un texte dans la mémoire de l'ordinateur revient alors à écrire une suite de nombres, donc une suite de 0 et de 1.

DÉFINITION Codification des caractères

La **codification des caractères** est l'opération qui consiste à associer à chaque caractère (lettre, chiffre, signe de ponctuation, symbole) un nombre entier unique, appelé son code, puis à représenter ce nombre en binaire pour qu'il soit manipulable par l'ordinateur. La table qui donne la correspondance entre les caractères et leurs codes s'appelle une table de codage.

À RETENIR L'idée fondamentale

Pour l'ordinateur, un texte n'est qu'une suite de nombres. Taper la lettre « A » au clavier, c'est en réalité enregistrer en mémoire le nombre 65, qui s'écrit 1000001 en binaire. Tout le reste de cette section n'est que l'application de cette idée.

5.2 Le code ASCII

Pour que tous les ordinateurs du monde se comprennent, il fallait une table de codage **commune et universelle**. C'est ainsi qu'est né, aux États-Unis en 1963, le code ASCII, encore utilisé aujourd'hui.

DÉFINITION Le code ASCII

Le code **ASCII** (sigle de American Standard Code for Information Interchange, c'est-à-dire « Code américain normalisé pour l'échange d'information ») est une table de codage qui associe un nombre à 128 caractères : les 26 lettres majuscules, les 26 lettres minuscules, les 10 chiffres, les signes de ponctuation, quelques symboles et des caractères de contrôle (comme le retour à la ligne). Les codes vont de 0 à 127, ce qui nécessite 7 bits ($2^7 = 128$) ; on les range habituellement dans un octet (8 bits) en ajoutant un 0 de poids fort.

PROPRIÉTÉ Codes ASCII à connaître

Quelques repères utiles dans la table ASCII :

- les chiffres « 0 » à « 9 » ont les codes 48 à 57 ;
- les lettres majuscules « A » à « Z » ont les codes 65 à 90 ;
- les lettres minuscules « a » à « z » ont les codes 97 à 122 ;
- l'espace a le code 32.

On remarque que les codes sont rangés dans l'ordre : B suit A, b suit a, « 1 » suit « 0 ». Pour passer d'une majuscule à la minuscule correspondante, on ajoute 32 (par exemple : $65 + 32 = 97$, donc A devient a).

EXEMPLE Trois codes à retenir par cœur

- La lettre A (majuscule) a pour code ASCII 65, soit 1000001 en binaire (sur 7 bits).
- La lettre a (minuscule) a pour code ASCII 97, soit 1100001 en binaire.
- Le chiffre 0 (le caractère « 0 ») a pour code ASCII 48, soit 0110000 en binaire.

Attention : le code 48 représente le caractère « 0 », pas le nombre zéro !

Caractère	Code décimal	Code binaire (7 bits)	Caractère	Code décimal	Code binaire (7 bits)
espace	32	0100000	M	77	1001101
0	48	0110000	N	78	1001110
1	49	0110001	O	79	1001111
3	51	0110011	a	97	1100001
A	65	1000001	b	98	1100010
B	66	1000010	c	99	1100011
C	67	1000011	m	109	1101101

Extrait de la table ASCII : chaque caractère possède un code décimal, que l'ordinateur écrit ensuite en binaire sur 7 bits.

5.3 Coder et décoder un mot avec la table ASCII

MÉTHODE Coder un mot en ASCII puis en binaire

Pour coder un mot, par exemple « CAM » :

1. Découper le mot en caractères : C, A, M.
2. Chercher dans la table ASCII le code décimal de chaque caractère : C = 67, A = 65, M = 77.
3. Convertir chaque code en binaire (par divisions successives par 2, comme on l'a appris dans la section précédente) : 67 = 1000011, 65 = 1000001, 77 = 1001101.
4. Écrire la suite des codes binaires les uns à la suite des autres : c'est le mot tel que l'ordinateur le mémorise.

Pour décoder, on fait le chemin inverse : on découpe la suite binaire en groupes de 7 bits, on convertit chaque groupe en décimal, puis on lit le caractère correspondant dans la table.

EXEMPLE Le mot « CAM » codé en ASCII

Le mot « CAM » se code, en décimal : 67 – 65 – 77, puis en binaire :

1000011 1000001 1001101

C'est cette suite de 21 bits (trois fois 7 bits) qui est réellement stockée dans la mémoire de l'ordinateur lorsque tu tapes « CAM ».

EXERCICE RÉSOLU 5 Coder puis décoder

Énoncé. (a) À l'aide de la table ASCII simplifiée ci-dessus, coder en décimal le mot « bac ». (b) Décoder la suite de codes : 66 – 79 – 78.

SOLUTION

Solution. (a) On découpe le mot : b, a, c. Dans la table : b = 98, a = 97, c = 99. Le mot « bac » se code donc 98 – 97 – 99. En binaire : 1100010 – 1100001 – 1100011.

(b) On cherche chaque code dans la table : 66 correspond à B, 79 correspond à O, 78 correspond à N. La suite 66 – 79 – 78 représente donc le mot « BON ».

ATTENTION Ne pas confondre le chiffre « 3 » et le nombre 3

C'est l'erreur la plus fréquente ! Le nombre 3 (une quantité, avec laquelle on calcule) se code en binaire : $3 = 11_2$. Mais le caractère « 3 » (le symbole que tu tapes au clavier dans

un texte) a pour code ASCII 51, soit 0110011 en binaire. Ce ne sont pas du tout les mêmes choses : dans « j'ai 3 stylos », le « 3 » est un caractère (code 51) ; dans un calcul $2 + 1 = 3$, le résultat est le nombre 3 (binaire 11). Retiens : tout ce qui est tapé au clavier est d'abord un caractère.

5.4 Les limites de l'ASCII et l'Unicode

Le code ASCII a été inventé aux États-Unis pour écrire l'anglais. Or l'anglais s'écrit sans aucun accent ! Avec seulement 128 caractères, impossible d'écrire correctement le **français** (é, è, à, ç), et encore moins les **langues camerounaises** qui utilisent des lettres spéciales, ni l'arabe, ni le chinois. Pour répondre à ce problème, on a d'abord créé des tables étendues (256 caractères sur 8 bits), puis une norme universelle : l'**Unicode**.

LESAVAIS-TU ?

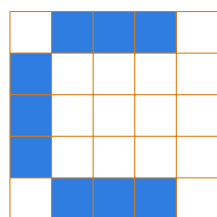
L'Unicode : écrire toutes les langues du monde

L'Unicode est une table de codage universelle qui attribue un numéro à plus de 140 000 caractères, couvrant presque toutes les écritures du monde : alphabet latin, arabe, chinois, cyrillique... Grâce à l'Unicode, on peut écrire sur ordinateur les lettres propres aux langues camerounaises, comme « ə » (présent en ewondo et en fulfuldé) ou « ŋ » (utilisé en ghoma-la' et en médumba). Ainsi, un élève de Bafoussam ou de Maroua peut écrire correctement son nom de famille ou un texte dans sa langue sur un ordinateur. Les premiers caractères de l'Unicode sont exactement ceux de l'ASCII : la lettre A garde le code 65, ce qui assure la compatibilité avec les anciens documents.

5.5 Codification des images et des sons : l'idée générale

Le principe « associer un nombre à chaque élément » ne s'arrête pas au texte. Il s'applique à toutes les données.

Les images. Une image numérique est découpée en une grille de minuscules carrés appelés **pixels** (contraction de picture elements, « éléments d'image »). À chaque pixel, on associe un nombre qui indique sa couleur : par exemple 0 pour un pixel noir et 1 pour un pixel blanc dans une image en noir et blanc. Une image devient ainsi un grand tableau de nombres, donc une suite de 0 et de 1. Plus il y a de pixels, plus l'image est précise : c'est ce qu'on appelle la **résolution**. Pour les images en couleur, on utilise généralement trois nombres par pixel (rouge, vert, bleu – codage RVB).



0 1 1 1 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
0 1 1 1 0

image agrandie : les pixels

1 = pixel coloré, 0 = pixel blanc

Une image en noir et blanc, fortement agrandie : chaque pixel est codé par un bit (1 = coloré, 0 = blanc). L'image devient une suite de 0 et de 1.

Lessons. Un son est une vibration continue de l'air. Pour le stocker, on mesure l'intensité du son à intervalles réguliers (par exemple des milliers de fois par seconde) : c'est l'**échantillonnage**. Chaque mesure est un nombre, que l'on code en binaire. Un morceau de musique devient ainsi une très longue suite de nombres. Pour écouter le son, l'ordinateur refait le chemin inverse : il transforme les nombres en vibrations du haut-parleur. La qualité dépend de la fréquence d'échantillonnage (nombre de mesures par seconde) et de la précision de chaque mesure (nombre de bits).

À RETENIR Un principe unique pour toutes les données

Texte, image, son, vidéo : dans tous les cas, la démarche est la même.

1. Découper la donnée en éléments simples (caractères, pixels, échantillons sonores).
2. Associer un nombre à chaque élément.
3. Coder ces nombres en binaire.

C'est pourquoi on dit que l'ordinateur est une machine numérique : tout, absolument tout, y est représenté par des nombres écrits en binaire.

EXERCICE 4 À toi de jouer

1. À l'aide de la table ASCII simplifiée de cette leçon, coder en décimal puis en binaire le mot « ON ».
2. Décoder la suite de codes ASCII : 67 – 79 – 78.
3. Expliquer, en deux phrases, pourquoi le code ASCII seul ne suffit pas pour écrire un texte en langue ewondo, et quelle solution existe.

CORRIGÉ Exercice

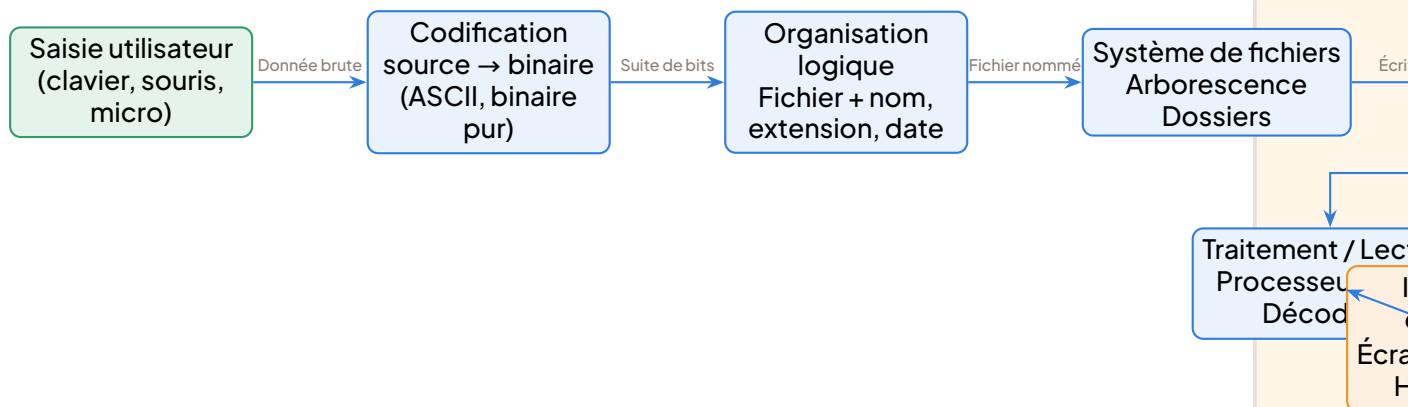
1. O = 79 et N = 78. Le mot « ON » se code 79 – 78 en décimal, soit 1001111 – 1001110 en binaire (sur 7 bits).
2. 67 = C, 79 = O, 78 = N : la suite représente le mot « CON ».
3. L'ASCII ne contient que 128 caractères pensés pour l'anglais : il ne possède ni les accents du français ni les lettres spéciales des langues camerounaises comme « ə ». La solution est d'utiliser l'Unicode, table universelle qui attribue un code à plus de 140 000 caractères de toutes les écritures du monde.

6**SYNTHÈSE ET APPLICATIONS À LA VIE COURANTE**

Nous arrivons au terme de cette leçon consacrée à l'organisation et à la codification des données. Dans les sections précédentes, nous avons disséqué chaque maillon de la chaîne : nous savons distinguer une **donnée** brute d'une **information** traitée (section 1), nous maîtrisons l'organisation hiérarchique en **fichiers**, **dossiers** et **arborescence** (section 2), nous comprenons que l'ordinateur ne manipule en fin de compte que des **bits** groupés en **octets** (section 3), et nous avons appris à **coder** les nombres entiers et les caractères (code ASCII) en binaire (sections 4 et 5).

Il est temps maintenant de **reassembler le puzzle** : comment ces notions s'enchaînent-elles dans la réalité d'un système informatique, du moment où tu appuies sur une touche du clavier jusqu'à l'affichage du résultat à l'écran ou au stockage sur une clé USB ? C'est l'objet de cette synthèse, illustrée par trois situations concrètes inspirées de ton quotidien au Cameroun.

6.1 Bilan : de la donnée brute au fichier binaire stocké



La chaîne complète du traitement de l'information : de la saisie au stockage, puis de la lecture à l'exploitation.

À RETENIR La chaîne complète : Saisie → Codification → Organisation → Stockage → Traitement

Chaque donnée manipulée par l'ordinateur suit un parcours en cinq étapes :

1. Saisie (Entrée) : un périphérique (clavier, scanner, microphone) capture une information du monde réel et la convertit en signal numérique brut.
2. Codification (Encodage) : le système traduit ce signal brut selon une convention standardisée (binaire pur pour un entier, ASCII pour un caractère). C'est l'étape Donnée → Suite de bits.
3. Organisation logique (Fichier) : la suite de bits est regroupée dans une entité nommée : le **fichier**. On y adjoint des informations descriptives (nom, extension, date de création, taille). L'extension (.txt, .jpg, .mp3) indique comment redécoder les bits.
4. Stockage physique : le système de fichiers place le fichier dans une arborescence de dossiers et écrit ses octets sur le support (disque dur, clé USB, carte SD).
5. Traitement / Restitution (Sortie) : pour être utile, le fichier est lu (chargé en mémoire vive), décodé (opération inverse de l'étape 2) par le processeur sous contrôle d'un logiciel, et présenté comme **information** à l'utilisateur (texte à l'écran, son dans les enceintes, image imprimée).

Règle d'or : Sans organisation (nom, dossier, extension), une suite de bits sur un disque est inutilisable. Sans codage standard, l'échange entre machines est impossible.

6.2 Méthode générale : résoudre un problème d'organisation et de codification

MÉTHODE Démarche structurée pour un projet de gestion de données

Face à une situation réelle (sauvegarder des photos, classer des factures, envoyer un message codé), suis ces 5 étapes pour produire une solution rigoureuse et justifiée :

1. Analyser le besoin (Problème) :

- Quelle est la nature des données ? (Texte, nombres, images, sons, mixte)
- Quel est le volume estimé ? (Nombre d'éléments × taille unitaire)
- Quelles sont les contraintes ? (Capacité du support, rapidité d'accès, partage, durée de conservation)
- Quel est l'objectif final ? (Archivage, consultation fréquente, envoi, impression)

2. Choisir l'organisation logique (Structure) :

- Définir une règle de nommage claire (ex : AAAA-MM-JJ_Sujet_Type.ext).
- Concevoir une arborescence de dossiers pertinente (par date, par projet, par type, par personne).
- Choisir les formats de fichiers (extensions) adaptés : .pdf pour les documents figés, .jpg pour les photos, .txt pour les textes simples.

3. Déterminer la codification (Encodage) :

- Pour du texte : code **ASCII** (caractères courants) ou ses extensions pour les accents.
- Pour des nombres entiers : binaire pur (conversion vue en section 4).
- Vérifier la taille résultante (poids du fichier) après codage.

4. Valider la faisabilité (Vérification quantitative) :

- Calculer la taille totale nécessaire = somme des tailles de chaque fichier, plus une marge (≈ 10%) pour le système de fichiers.
- Comparer à la capacité réelle disponible du support (attention : une clé de 16 Go n'offre que ≈ 14,9 Go utilisables, voir plus loin).
- Si Taille totale > Capacité disponible → revoir l'organisation (sélection, support plus grand).

5. Exécuter, tester et documenter :

- Effectuer la copie / l'envoi / la création.
- Tester la lecture (ouvrir 2 ou 3 fichiers au hasard) pour valider le décodage.
- Rédiger un rapport court : choix faits, volume final, support utilisé, date, problèmes rencontrés. C'est la traçabilité.

6.3 Application 1 : organiser les fichiers d'un cybercafé (factures, photos, devoirs)

Imaginons **Paul**, gérant du cybercafé « Connexion Express » à Yaoundé, quartier Mvog-Ada. Il doit gérer trois types de documents numériques sur le PC administratif (disque de 500 Go) :

- **Factures clients** (PDF, environ 200 Ko chacune, environ 50 par mois).
- **Photos d'événements** (anniversaires, tournois de jeux vidéo) : JPEG, environ 3 Mo chacune, environ 200 par mois.
- **Devoirs d'élèves** (Word/PDF, environ 500 Ko, environ 100 par mois) imprimés sur place.

EXEMPLE Arborescence proposée pour le cybercafé

Paul applique la méthode (étape 2). Il choisit une organisation par année/mois/type, la plus naturelle pour une gestion administrative chronologique.

```
D:\CyberData\
+--- 2025\
|   +--- 01_Janvier\
|       +--- Factures\
|           +--- 2025-01-02_Facture_001_Client_Ngon.pdf
|           +--- 2025-01-15_Facture_002_Client_Mbia.pdf
|       +--- Photos_Evenements\
|           +--- 2025-01-05_Tournoi_Jeux_Equipe_A.jpg
|           +--- 2025-01-20_Anniv_Petite_Soeur.jpg
|       +--- Devoirs_Eleves\
|           +--- 2025-01-10_Devoir_Math_3eme_Kamga.docx
|           +--- 2025-01-22_Devoir_Physique_Tle_Ondoa.pdf
|   +--- 02_Fevrier\
|       +--- ...
+--- Archives_Anterieures\
    +--- 2024\
        +--- ...
```

Justification des choix :

- **Nommage** : AAAA-MM-JJ_Objet_ID_Auteur.ext → le tri alphabétique correspond au tri chronologique ; l'auteur ou le client est identifiable ; l'extension est claire.
- **Dossiers par mois** : évite d'avoir des milliers de fichiers dans un même dossier (ce qui ralentit l'explorateur et les recherches).
- **Séparation par type** : permet des sauvegardes différenciées (Factures = critiques → double sauvegarde ; Photos = volumineuses → disque externe ; Devoirs = temporaires → suppression après impression).

EXERCICE RÉSOLU 6 Estimation mensuelle de l'espace disque

Énoncé : Calculer l'espace disque mensuel nécessaire pour l'activité du cybercafé. Le disque de 500 Go est-il suffisant pour un an d'activité sans nettoyage ?

SOLUTION

Solution détaillée :

1. Volume mensuel par catégorie :

Factures : $50 \times 200 \text{ Ko} = 10\,000 \text{ Ko} \approx 10 \text{ Mo}$

Photos : $200 \times 3 \text{ Mo} = 600 \text{ Mo}$

Devoirs : $100 \times 500 \text{ Ko} = 50\,000 \text{ Ko} \approx 50 \text{ Mo}$

2. Total mensuel brut : $10 + 600 + 50 = 660 \text{ Mo}$.

3. Avec marge système de fichiers (10 %) : $660 \times 1,1 = 726 \text{ Mo/mois}$.

4. Total annuel estimé : $726 \times 12 = 8\,712 \text{ Mo} \approx 8,7 \text{ Go}$.

5. Conclusion : le disque de 500 Go est largement suffisant pour plusieurs années. Le vrai problème n'est donc pas la capacité, mais l'organisation pour retrouver un fichier rapidement.

6.4 Application 2 : estimer l'espace pour sauvegarder les photos d'un mariage

Situation classique : **Chantal** a photographié le mariage de sa cousine à Douala. Elle a **500 photos** de **25 Mo** chacune sur la carte SD de son appareil. Elle veut les sauvegarder sur sa clé USB de **16 Go** avant de vider la carte.

EXEMPLE Calcul complet : 500 photos sur une clé de 16 Go

Données :

- Nombre de photos : $N = 500$.
- Taille unitaire : $T_u = 25 \text{ Mo}$.
- Capacité annoncée de la clé : $C = 16 \text{ Go}$.

Étape 1 : Volume total des données

$$V = N \times T_u = 500 \times 25 \text{ Mo} = 12\,500 \text{ Mo} = 12,5 \text{ Go}.$$

Étape 2 : Capacité réellement utilisable de la clé

Les fabricants comptent en puissances de 10 ($1 \text{ Go} = 10^9 \text{ octets}$), mais l'ordinateur compte en puissances de 2 ($1 \text{ Gio} = 2^{30} \text{ octets} = 1\,073\,741\,824 \text{ octets}$). La capacité vue par l'ordinateur est donc :

$$C_{\text{réelle}} = \frac{16 \times 10^9}{2^{30}} \approx \frac{16\,000\,000\,000}{1\,073\,741\,824} \approx 14,9 \text{ Gio}.$$

Le système de fichiers consomme encore une petite part ; on dispose d'environ 14,7 Gio.

Étape 3 : Comparaison (dans les mêmes unités)

Convertissons le volume des données dans l'unité de l'ordinateur :

$$V = \frac{12,5 \times 10^9}{2^{30}} \approx 11,6 \text{ Gio.}$$

Résultat : $11,6 \text{ Gio} < 14,7 \text{ Gio} \rightarrow$ la copie est possible ! Il reste environ 3 Gio de marge.

Étape 4 : Et si Chantal compressait ses photos (5 Mo/photo) ?

$$V' = 500 \times 5 \text{ Mo} = 2\,500 \text{ Mo} = 2,5 \text{ Go} \approx 2,3 \text{ Gio.}$$

Gain de place énorme, mais avec une perte de qualité. Le choix dépend de l'usage : impression grand format $\rightarrow$ garder les fichiers d'origine ; partage sur WhatsApp $\rightarrow$ la version compressée suffit.

ATTENTION Erreurs fréquentes : unités et limites des supports

- Piège n°1 : acheter une clé « 16 Go » en pensant pouvoir y mettre 16 Gio de données. Faux. On ne pourra y mettre que $\approx 14,9 \text{ Gio}$. Règle pratique : Capacité réelle $\approx$ Capacité annoncée $\times 0,93$.
- Piège n°2 : comparer directement 12 500 Mo à « 16 Go » sans conversion. Ici la copie tient quand même, mais avec une clé « 8 Go » ($\approx 7,45 \text{ Gio}$ réels), $11,6 > 7,45 \rightarrow$ échec de la copie en cours de route.
- Piège n°3 : oublier de laisser de la marge. Bonne pratique : toujours laisser 10 à 15 % d'espace libre sur une clé USB ou une carte SD pour garantir son bon fonctionnement.
- Piège n°4 : oublier de vérifier après la copie : ouvrir quelques fichiers copiés avant d'effacer les originaux de la carte SD !

6.5 Application 3 : transmettre un message codé en binaire entre élèves

Activité ludique mais rigoureuse : **Alice** et **Bob**, élèves en 3ème au Lycée de Biyem-Assi, veulent s'échanger un court message secret pendant l'heure d'informatique (sur papier, sans téléphone), en utilisant ce qu'ils ont appris sur le code ASCII.

EXPÉRIENCE TP guidé : échange d'un message binaire « main dans la main »

Objectif : mettre en œuvre la chaîne complète : Texte $\rightarrow$ Codage ASCII (binaire) $\rightarrow$ Transmission (papier) $\rightarrow$ Décodage $\rightarrow$ Texte.

Matériel : papier, crayon, table ASCII (extrait fourni ci-dessous).

Manipulation :

1. Accord préalable (protocole) : Alice et Bob conviennent :

- Codage : ASCII sur 8 bits par caractère (on complète le code sur 7 bits par un 0 à gauche).
- Séparateur : un espace entre les octets (pour la lisibilité humaine).
- Fin de message : la séquence 00000000.

2. Écriture (Alice – Encodage) : Alice choisit le mot SALUT.

- Elle consulte la table : S = 83, A = 65, L = 76, U = 85, T = 84.
- Elle convertit chaque décimal en binaire sur 8 bits (méthode des puissances de 2) :

$$83 = 64 + 16 + 2 + 1 = 01010011$$

$$65 = 64 + 1 = 01000001$$

$$76 = 64 + 8 + 4 = 01001100$$

$$85 = 64 + 16 + 4 + 1 = 01010101$$

$$84 = 64 + 16 + 4 = 01010100$$

- Elle écrit sur le papier : 01010011 01000001 01001100 01010101 01010100 00000000.

3. Transmission : Alice passe le papier à Bob.

4. Lecture (Bob – Décodage) : Bob groupe les bits par 8, convertit chaque octet en décimal, puis retrouve le caractère dans la table.

5. Vérification : Bob lit « SALUT ». Succès!

Table de référence (extrait du code ASCII) :

Caractère	Décimal	Binaire (8 bits)	Caractère	Décimal	Binaire (8 bits)
Espace	32	00100000	A	65	01000001
B	66	01000010	C	67	01000011
L	76	01001100	S	83	01010011
T	84	01010100	U	85	01010101
a	97	01100001	b	98	01100010
O	48	00110000	1	49	00110001
2	50	00110010	5	53	00110101
!	33	00100001	?	63	00111111

Observations et interprétation :

- Rigidité du protocole : si Alice oublie un espace entre deux octets, Bob ne sait plus où couper la suite de bits : le message devient ambigu.
- Coût : 5 caractères + fin de message → 48 bits à écrire à la main. C'est long! C'est pourquoi ce travail est confié aux machines.
- Fragilité : une erreur sur un seul bit change complètement la lettre : 01010011 (S) devient 01010010 (R). D'où l'importance de la concentration... et des contrôles!

6.6 Rédaction d'une démarche justifiée : du problème à la vérification

Le programme insiste sur le **savoir-faire** : « Rédiger et justifier une démarche, une réponse ou une conclusion ». Voici le modèle type attendu au BEPC pour une question de synthèse sur l'organisation et la codification des données.

MÉTHODE

Structure type d'une réponse justifiée (épreuve écrite)

Pour toute question du type « Proposez une organisation pour... » ou « Est-il possible de stocker... ? Justifiez. », structure ta réponse en 4 étapes distinctes :

1. **Problème et données (état des lieux)** : reprends les chiffres clés de l'énoncé.
« Le problème est de sauvegarder 500 photos de 25 Mo chacune sur une clé USB annoncée à 16 Go. Données : $N = 500$, $T_u = 25 \text{ Mo}$, $C = 16 \text{ Go}$. »
2. **Choix d'organisation et de codage (stratégie)** : explique pourquoi tu choisis tel format, tel nommage, telle arborescence.
« Je choisis de conserver les fichiers d'origine (pas de perte de qualité) pour l'archivage. Les fichiers seront placés dans un dossier unique *2025-06-15_Mariage* et nommés *Mariage_001.jpg*, *Mariage_002.jpg*, etc. »
3. **Calculs et vérification quantitative (preuve)** : montre les formules, les conversions d'unités, la comparaison.
« Volume total $V = 500 \times 25 = 12\,500 \text{ Mo} = 12,5 \text{ Go}$. Capacité réelle de la clé : $16 \times 0,93 \approx 14,9 \text{ Gio}$. Conversion : $V \approx 11,6 \text{ Gio}$. Comme $11,6 < 14,9$, la sauvegarde est possible avec environ 3 Gio de marge. »
4. **Conclusion et recommandations (synthèse)** : réponse finale claire + conseils.
« Conclusion : la clé de 16 Go convient. Recommandation : copier le dossier, vérifier l'ouverture de 3 photos au hasard, puis éjecter la clé proprement avant de la retirer. »

EXERCICE RÉSOLU 7

Sujet type BEPC : organisation des bulletins de notes

Énoncé : L'administration du Lycée de Nkolbisson doit archiver les bulletins de notes de 600 élèves (3 trimestres, soit 3 bulletins par élève et par an) sur 5 ans. Chaque bulletin est un fichier PDF de 150 Ko. L'archive sera stockée sur un disque dur externe de 500 Go.

1. Proposer une arborescence de dossiers et une convention de nommage.
2. Calculer le volume total sur 5 ans. Vérifier la faisabilité.
3. Donner deux avantages de cette organisation pour la recherche ultérieure.

SOLUTION

Solution détaillée :

1. Organisation proposée :

```

D:\Archives_Bulletins\
+-- 2024-2025\
|   +-- 6eme\
|       +-- 6eme_A\
|           +-- 2024-2025_6eme_A_Abena_Paul_T1.pdf
|           +-- ...
|       +-- 6eme_B\
|   +-- 5eme\
|       +-- ...
+-- 2025-2026\
|   +-- ...
+-- 2028-2029\
|   +-- ...

```

Nommage : AnnéeScolaire_Classe_Nom_Prenom_Trimestre.pdf (ex : 2024-2025_3eme_C_Mvondo_Jean_T2.pdf).

Justification : hiérarchie Année → Niveau → Classe → Fichiers. Elle permet de supprimer une année complète facilement, ou de retrouver tous les bulletins d'un élève en cherchant son nom.

2. Calcul du volume :

Nombre de bulletins par an = $600 \times 3 = 1800$

Volume par an = $1800 \times 150 \text{ Ko} = 270\,000 \text{ Ko} \approx 270 \text{ Mo}$

Volume sur 5 ans = $5 \times 270 \text{ Mo} = 1350 \text{ Mo} \approx 1,35 \text{ Go}$

Avec marge de 10 % $\approx 1,5 \text{ Go}$

Le disque de 500 Go ($\approx 465 \text{ Gio}$ réels) est très largement suffisant : $1,5 \ll 465$. L'archivage est faisable sans difficulté.

3. Avantages pour la recherche :

1. Navigation intuitive : on retrouve le bulletin de Jean Mvondo en 3 clics (Année → 3ème → 3ème C) sans outil de recherche.
2. Recherche par nom : la règle de nommage régulière permet de retrouver rapidement, par exemple, tous les bulletins du trimestre T2 d'une classe donnée.

6.7 Le savais-tu ? : Le Mobile Money, une organisation de données géante dans ta poche

LES AVAIS-TU ?

MTN MoMo, Orange Money : la rigueur des données financières

Chaque fois qu'un utilisateur fait un transfert MTN MoMo ou Orange Money depuis son téléphone, une chaîne de traitement de données très structurée s'exécute en quelques secondes :

1. Saisie structurée : l'utilisateur tape le numéro du destinataire (9 chiffres), le montant (un

nombre entier en FCFA) et son code secret. Le téléphone vérifie le format de chaque donnée avant l'envoi.

2. Codification : la demande est convertie en une suite de bits et protégée (rendue illisible pour les autres) avant de voyager sur le réseau mobile.
3. Traitement par le serveur central : l'ordinateur de l'opérateur effectue trois opérations indissociables : retirer le montant du compte de l'expéditeur, l'ajouter au compte du destinataire, et enregistrer l'opération dans un historique (date, heure, numéros, montant).
4. Organisation du stockage : les données ne sont pas rangées dans de simples fichiers, mais dans des tableaux structurés (listes d'utilisateurs, de comptes, de transactions) conservés sur des serveurs sécurisés, avec des copies de sauvegarde.
5. Réponse (décodage) : le serveur renvoie le résultat ; le téléphone le decode et affiche : « Transfert de 5 000 FCFA effectué. Nouveau solde : 12 300 FCFA. »

Le lien avec notre leçon :

- Donnée brute → les touches tapées sur le téléphone.
- Codification → la conversion en binaire et la protection du message.
- Organisation → des tableaux de données bien structurés et nommés.
- Stockage → des serveurs avec sauvegardes, dans des centres informatiques sécurisés.
- Information → le message de confirmation que l'utilisateur lit.

Enjeu critique : une erreur de codage (un montant mal converti), une mauvaise organisation (recherche trop lente) ou une panne de stockage sans sauvegarde (perte de l'historique) sont inacceptables quand il s'agit d'argent. C'est pourquoi les opérateurs de Mobile Money recrutent des informaticiens rigoureux, formés dès le collège à ces concepts de base !

6.8 Exercices de synthèse (préparation BEPC)

EXERCICE 5 Exercice 1 : sauvegarde sélective

Tu disposes d'une carte SD contenant :

- 1200 photos JPEG (moyenne 4,5 Mo) ;
- 15 vidéos (moyenne 1,2 Go) ;
- 50 photos grand format (moyenne 28 Mo).

Tu veux tout copier sur un disque dur externe de 500 Go (déjà occupé à 320 Go).

1. Calcule l'espace total occupé sur la carte SD (en Go).

2. Calcule l'espace libre restant sur le disque dur (en Go).
3. La copie complète est-elle possible ? Justifie par le calcul.
4. Si l'espace libre n'était que de 6 Go, propose une stratégie pour sauvegarder le maximum de données prioritaires (Photos > Vidéos > Grand format).

EXERCICE 6 Exercice 2 : codage et transmission d'un mot de passe

Dans le cadre d'un projet de classe, vous devez transmettre le mot de passe Wi-Fi Lycee2025! à un camarade via un papier. Vous décidez de le coder en binaire à l'aide du code ASCII (1 octet par caractère).

1. Donne le code décimal ASCII de chaque caractère : L, y, c, e, e, 2, 0, 2, 5, ! (on donne : y = 121, c = 99, e = 101).
2. Convertis chaque décimal en binaire sur 8 bits.
3. Écris la trame complète à transmettre (octets séparés par un espace).
4. Combien de bits au total ? Si l'on commet une erreur sur 1 bit sur les 80, quel est le taux d'erreur (en pourcentage) ?

EXERCICE 7 Exercice 3 : organisation d'une médiathèque scolaire

Le club informatique du lycée veut numériser 2 000 fiches de lecture (texte + image de couverture). Chaque fiche = 1 page enregistrée en .pdf + 1 image JPEG (couverture).

- Page PDF : environ 80 Ko.
 - Couverture JPEG : environ 500 Ko.
1. Propose une arborescence de dossiers et un nommage normalisé pour ces 2 000 paires de fichiers.
 2. Estime le volume total en Mo puis en Go.
 3. Quelle clé USB minimale recommandes-tu (tailles standards : 8, 16, 32, 64 Go) ? Justifie avec une marge de 20 %.

CORRIGÉ Exercice 1

1. Volume de la carte SD :

$$\text{Photos} = 1200 \times 4,5 \text{ Mo} = 5400 \text{ Mo} = 5,4 \text{ Go}$$

$$\text{Vidéos} = 15 \times 1,2 \text{ Go} = 18 \text{ Go}$$

$$\text{Grand format} = 50 \times 28 \text{ Mo} = 1400 \text{ Mo} = 1,4 \text{ Go}$$

$$\text{Total} = 5,4 + 18 + 1,4 = 24,8 \text{ Go}$$

2. Espace libre sur le disque : $500 - 320 = 180 \text{ Go}$ (on néglige ici l'écart Go/Gio, identique pour les deux valeurs).
3. Faisabilité : $24,8 \text{ Go} < 180 \text{ Go} \rightarrow$ oui, la copie complète est possible, avec une large marge.
4. Stratégie avec 6 Go libres : priorité 1 : les photos JPEG (5,4 Go) $\rightarrow$ elles tiennent dans les 6 Go. Priorité 2 : les vidéos (18 Go) $\rightarrow$ impossibles, on n'en copie aucune en entier. Priorité 3 : le grand format (1,4 Go) $\rightarrow$ ne tient plus ($5,4 + 1,4 = 6,8 > 6$). Solution : copier toutes les photos JPEG (5,4 Go) + environ 20 fichiers grand format (0,56 Go) pour rester sous 6 Go, et reporter les vidéos sur un autre support.

CORRIGÉ Exercice 2

1. Codes décimaux ASCII : L = 76, y = 121, c = 99, e = 101, e = 101, 2 = 50, 0 = 48, 2 = 50, 5 = 53, ! = 33.

2. Conversion en binaire sur 8 bits :

$$76 = 64 + 8 + 4 = 01001100$$

$$121 = 64 + 32 + 16 + 8 + 1 = 01111001$$

$$99 = 64 + 32 + 2 + 1 = 01100011$$

$$101 = 64 + 32 + 4 + 1 = 01100101$$

$$101 = 01100101$$

$$50 = 32 + 16 + 2 = 00110010$$

$$48 = 32 + 16 = 00110000$$

$$50 = 00110010$$

$$53 = 32 + 16 + 4 + 1 = 00110101$$

$$33 = 32 + 1 = 00100001$$

3. Trame complète :

01001100 01111001 01100011 01100101 01100101 00110010 00110000 00110010
00110101 00100001

4. Total : $10 \times 8 = 80 \text{ bits}$. Taux d'erreur : $\frac{1}{80} = 0,0125 = 1,25 \%$.

CORRIGÉ Exercice 3

1. Arborescence proposée :

```
Mediatheque_Lycee\  
+-- Fiches_PDF\  
|  
| +-- 6eme\  
| | +-- 6eme_Abena_Paul_Fiche.pdf  
| | +-- ...  
| +-- 5eme\  
| +-- ...  
+-- Couvertures_JPG\  
| +-- 6eme\  
| | +-- 6eme_Abena_Paul_Couv.jpg  
| | +-- ...  
+-- 5eme\  
+-- ...
```

Nommage : Classe_Nom_Prenom_Type.ext (Type = Fiche ou Couv). La fiche et sa couverture sont liées par un nom de base commun.

2. Volume total :

$$\begin{aligned}\text{Fiches} &= 2\,000 \times 80 \text{ Ko} = 160\,000 \text{ Ko} \approx 160 \text{ Mo} \\ \text{Couvertures} &= 2\,000 \times 500 \text{ Ko} = 1\,000\,000 \text{ Ko} \approx 1\,000 \text{ Mo} \\ \text{Total} &\approx 1\,160 \text{ Mo} \approx 1,16 \text{ Go}\end{aligned}$$

3. Support : avec une marge de 20 % : $1,16 \times 1,2 \approx 1,4$ Go nécessaires. Une clé de 8 Go (environ 7,4 Go réels) suffit largement. Recommandation : une clé de 16 Go, au prix très proche, offre une marge confortable pour les ajouts futurs de la médiathèque.

À RETENIR Check-list finale : suis-je prêt pour le BEPC sur cette leçon ?

- ☐ Je sais distinguer donnée, information et type de donnée (entier, réel, caractère, chaîne de caractères).
- ☐ Je sais organiser une arborescence logique, nommée et hiérarchique adaptée à un contexte (cybercafé, école, usage personnel).
- ☐ Je connais les unités : bit, octet, Ko, Mo, Go, To, et je sais convertir de l'une à l'autre (1 Ko = 1 024 octets en binaire, 1 Mo = 1 024 Ko, etc.).
- ☐ Je sais que la capacité réelle d'un support est inférieure à la capacité annoncée (règle pratique : $\times 0,93$).
- ☐ Je sais coder un entier positif en binaire (méthode des puissances de 2) et décoder un nombre binaire.

- ☐ Je connais le code ASCII : chaque caractère possède un numéro, codé sur 7 bits (128 caractères), que l'on écrit sur 8 bits en ajoutant un 0.
- ☐ Je sais qu'une image est un tableau de points (pixels) codés en binaire, et qu'un son est une suite de valeurs numériques.
- ☐ J'applique la méthode en 5 étapes (Analyser, Organiser, Coder, Vérifier, Exécuter/-Tester) pour tout problème concret.
- ☐ Je sais rédiger une démarche justifiée : Problème → Choix → Calculs/Preuve → Conclusion/Recommandation.

Bravo! Tu as maintenant les bases solides pour comprendre comment l'ordinateur « pense » en binaire et comment nous, humains, organisons ce monde binaire en informations utiles. Les prochaines leçons te montreront comment traiter ces données automatiquement grâce aux algorithmes. Courage pour la suite!

7 ACTIVITÉ D'INTÉGRATION

8 LEÇON 4 : ORGANISATION DES DONNÉES EN INFORMATIQUE

8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu devras être capable de :

- distinguer **donnée**, **information** et **type de données** dans une situation réelle de la vie courante ;
- transformer des données brutes (écrites à la main, désordonnées) en un **tableau structuré** dont les colonnes sont des champs bien définis ;
- proposer une **arborescence de dossiers et de fichiers** cohérente, avec des noms de fichiers corrects ;
- **coder un nombre entier en binaire** et **coder un texte en ASCII** ;
- calculer l'**espace disque** nécessaire au stockage de fiches, en utilisant les unités de mesure de l'information (Ko, Mo) ;
- rédiger et justifier une démarche, une réponse ou une conclusion, comme l'exige le programme.

8.2 Situation

Au marché Mokolo, l'un des plus grands marchés de Yaoundé, Madame Béatrice tient un étal de produits vivriers depuis quinze ans. Chaque soir, elle note ses ventes à la main dans un cahier, sans ordre précis. Voici un extrait fidèle de ses notes pour la semaine du lundi 6 au samedi 11 janvier 2025 :

EXEMPLE Le cahier de notes de Madame Béatrice (données brutes)

- « Lundi 6/01 : 4 kg de tomates à 500 FCFA le kg ; 2 sacs d'oignons à 1 000 FCFA le sac. »
- « Mardi 7/01 : 6 kg de tomates à 500 FCFA ; 3 kg de piment à 800 FCFA le kg. »
- « Mercredi 8/01 : 5 kg de tomates à 500 FCFA ; 1 sac d'oignons à 1 000 FCFA ; 2 kg de piment à 800 FCFA. »
- « Jeudi 9/01 : 3 kg de tomates à 500 FCFA ; 4 kg de piment à 800 FCFA. »
- « Vendredi 10/01 : 7 kg de tomates à 500 FCFA ; 2 sacs d'oignons à 1 000 FCFA. »

- « Samedi 11/01 : 8 kg de tomates à 500 FCFA ; 3 sacs d'oignons à 1 000 FCFA ; 1 kg de piment à 800 FCFA. »

Le fils de Madame Béatrice, élève en classe de 3ème, vient de recevoir un ordinateur offert par son oncle. Il propose à sa mère de créer un cahier numérique pour remplacer le cahier papier. Mais il se pose plusieurs questions : comment ranger toutes ces ventes dans l'ordinateur ? Comment l'ordinateur va-t-il représenter les nombres et les noms de produits ? Combien de place faudra-t-il sur le disque dur ?

Données numériques supplémentaires :

- Madame Béatrice vend 3 produits différents : tomates, oignons, piment.
- Elle estime créer 1500 fiches produits par an, chaque fiche occupant 2 Ko sur le disque.
- Pour le codage ASCII, on donne : A = 65, C = 67, E = 69, I = 73, M = 77, O = 79, S = 83, T = 84 (codes décimaux des lettres majuscules utiles).

8.3 Consignes

Par groupes de 4 élèves, répondez aux questions suivantes en rédigeant soigneusement vos démarches. Chaque réponse doit être **justifiée**.

1. **Trier et structurer.** À partir du cahier de notes, construisez un tableau structuré des ventes de la semaine. Chaque ligne représentera une vente et chaque colonne un champ : Date, Produit, Quantité, Unité, Prix unitaire (FCFA), Montant (FCFA). Pour chaque champ, indiquez le **type de données** choisi (texte, nombre entier, date) et justifiez votre choix.
2. **Calculer.** Ajoutez une colonne « Montant » et calculez le montant de chaque vente ($\text{Montant} = \text{Quantité} \times \text{Prix unitaire}$), puis le montant total de la semaine.
3. **Ranger.** Proposez une arborescence de dossiers et de fichiers pour ranger les ventes de toute l'année, par mois et par produit. Donnez trois exemples de noms de fichiers corrects (sans espace, sans accent, sans caractère spécial interdit) et expliquez pourquoi ces règles de nommage sont importantes.
4. **Coder les nombres.** Calculez le nombre total de produits vendus dans la semaine (toutes quantités confondues), puis convertissez ce nombre en binaire en détaillant la méthode des divisions successives par 2.
5. **Coder les caractères.** Identifiez le produit le plus vendu de la semaine, puis codez son nom en ASCII (en majuscules) à l'aide des codes fournis. Donnez la suite des codes décimaux obtenue.
6. **Estimer l'espace disque.** Calculez l'espace disque nécessaire pour stocker les 1500 fiches de l'année, en Ko puis en Mo. Madame Béatrice dispose d'une clé USB de 4 Go : cet espace est-il suffisant ? Justifiez.
7. **Rédiger la conclusion.** Rédigez un court texte (10 à 15 lignes) adressé à Madame Béatrice, expliquant chaque choix effectué (structure du tableau, arborescence, codage) et les avantages du cahier numérique par rapport au cahier papier.

Restitution : chaque groupe présente ses résultats à l'oral (5 minutes). La correction collective est construite au tableau avec l'enseignant.

8.4 Ressources à mobiliser

À RETENIR Boîte à outils de la leçon

- Types de données : texte (chaîne de caractères), nombre entier, nombre décimal, date, booléen.
- Organisation structurée : un tableau = des lignes (enregistrements) et des colonnes (champs); une arborescence = des dossiers qui contiennent des sous-dossiers et des fichiers.
- Noms de fichiers : pas d'espace, pas d'accent, pas de caractères spéciaux (\ / : * ? " < > |); on préfère les tirets ou les tirets bas.
- Conversion décimal → binaire : divisions successives par 2; on lit les restes de bas en haut.
- Vérification : un nombre binaire $(b_n \dots b_1 b_0)_2$ vaut $\sum b_i \times 2^i$.
- Codage ASCII : chaque caractère est représenté par un nombre (par exemple A = 65).
- Unités de stockage : 1 Ko = 1024 octets; 1 Mo = 1024 Ko; 1 Go = 1024 Mo.

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 8 Le cahier numérique du marché Mokolo — corrigé détaillé

Énoncé : structurer les ventes de la semaine, proposer une arborescence, coder le total en binaire et le produit le plus vendu en ASCII, estimer l'espace disque, puis justifier chaque choix.

SOLUTION

Étape 1 — Tableau structuré des ventes. On dépouille le cahier ligne par ligne. Chaque vente devient un enregistrement :

Date	Produit	Qté	Unité	P.U. (F)	Montant (F)
06/01/2025	Tomates	4	kg	500	2000
06/01/2025	Oignons	2	sac	1000	2000
07/01/2025	Tomates	6	kg	500	3000
07/01/2025	Piment	3	kg	800	2400
08/01/2025	Tomates	5	kg	500	2500
08/01/2025	Oignons	1	sac	1000	1000
08/01/2025	Piment	2	kg	800	1600
09/01/2025	Tomates	3	kg	500	1500
09/01/2025	Piment	4	kg	800	3200
10/01/2025	Tomates	7	kg	500	3500
10/01/2025	Oignons	2	sac	1000	2000
11/01/2025	Tomates	8	kg	500	4000
11/01/2025	Oignons	3	sac	1000	3000
11/01/2025	Piment	1	kg	800	800

Types de données justifiés : Date : type date (pour trier chronologiquement) ; Produit et Unité : type texte ; Quantité et Prix unitaire : type nombre entier (pour faire des calculs) ; Montant : nombre entier calculé.

Étape 2 — Montant total de la semaine.

$2000+2000+3000+2400+2500+1000+1600+1500+3200+3500+2000+4000+3000+800 = 32500$ FC

Étape 3 — Arborescence proposée. On crée un dossier principal `VENTES_2025`, contenant un sous-dossier par mois, lui-même contenant un fichier par produit :

```

VENTES_2025/
|-- JANVIER/
|   |-- ventes_tomates_janvier.txt
|   |-- ventes_oignons_janvier.txt
|   |-- ventes_piment_janvier.txt
|-- FEVRIER/
|   |-- ventes_tomates_fevrier.txt
|   |-- ...

```

Justification : les noms sont en minuscules, sans espace ni accent, et décrivent le contenu ; on retrouve ainsi un fichier en quelques secondes, même après plusieurs mois.

Étape 4 — Codage binaire du total. Quantités vendues : tomates $4 + 6 + 5 + 3 + 7 + 8 = 33$; oignons $2 + 1 + 2 + 3 = 8$; piment $3 + 2 + 4 + 1 = 10$. Total : $33 + 8 + 10 = 51$ produits. Divisions successives par 2 :

$$51 = 2 \times 25 + 1$$

$$6 = 2 \times 3 + 0$$

$$25 = 2 \times 12 + 1$$

$$3 = 2 \times 1 + 1$$

$$12 = 2 \times 6 + 0$$

$$1 = 2 \times 0 + 1$$

On lit les restes de bas en haut : $51 = (110011)_2$. Vérification : $1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 32 + 16 + 2 + 1 = 51$. Correct !

Étape 5 — Codage ASCII du produit le plus vendu. Le produit le plus vendu est la tomate (33 unités). En majuscules : TOMATES. Avec les codes fournis :

T	O	M	A	T	E	S
84	79	77	65	84	69	83

L'ordinateur stocke donc « TOMATES » comme la suite de nombres 84, 79, 77, 65, 84, 69, 83, chacun étant lui-même codé en binaire.

Étape 6 — Espace disque.

$$1500 \text{ fiches} \times 2 \text{ Ko} = 3000 \text{ Ko} = \frac{3000}{1024} \approx 2,93 \text{ Mo.}$$

La clé USB de 4 Go (= 4096 Mo) est très largement suffisante : elle pourrait contenir plus de mille années d'archives à ce rythme !

Étape 7 — Texte de justification (exemple attendu). « Chère maman, nous avons rangé tes ventes dans un tableau où chaque colonne a un rôle précis : la date, le produit, la quantité, le prix et le montant. Ainsi, l'ordinateur peut calculer et trier automatiquement. Les fichiers sont rangés dans des dossiers par mois et par produit, avec des noms sans espace ni accent pour éviter les erreurs. Dans la mémoire de l'ordinateur, le nombre 51 s'écrit 110011 en binaire, et le mot TOMATES est remplacé par des nombres grâce au code ASCII. Toutes tes fiches de l'année n'occuperont que 3 Mo environ : ta clé USB suffit largement. Le cahier numérique évite les ratures, les pertes et les erreurs de calcul. »

ATTENTION

Erreurs fréquentes à signaler lors de la correction

- Lire les restes de la division de haut en bas au lieu de bas en haut : on obtiendrait $(110011)_2$ lu à l'envers, c'est-à-dire une erreur si le nombre n'est pas un palindrome. Toujours vérifier par la somme des puissances de 2.
- Confondre Ko et Mo : diviser par 1024, pas par 1000.
- Utiliser des espaces ou des accents dans les noms de fichiers (ventes piment janvier.txt est à proscrire).
- Oublier de justifier le type de chaque champ : c'est pourtant ce qui permet à l'ordinateur de traiter correctement les données.

8.6 Bilan de l'activité

Cette activité a permis de mettre en évidence, à partir d'une situation authentique du marché Mokolo, la **chaîne complète de traitement des données** étudiée dans la leçon :

- des **données brutes** (notes manuscrites désordonnées) ne deviennent de véritables **informations** exploitables qu'après avoir été **structurées** en champs et en enregistrements, chaque champ recevant un **type de données** adapté ;
- l'**organisation en arborescence** de dossiers et de fichiers, avec des règles de nommage strictes, garantit que l'on retrouve rapidement une information, même en grande quantité ;
- à l'intérieur de l'ordinateur, **toute donnée est codée en binaire** : les nombres par conversion en base 2, les caractères grâce à des tables comme le code ASCII ;
- enfin, le calcul de l'**espace de stockage** montre concrètement l'intérêt des unités de mesure de l'information (octet, Ko, Mo, Go) pour anticiper les besoins matériels.

Tu as ainsi mobilisé les deux savoir-faire officiels de la leçon : appliquer les notions de l'unité à une situation concrète de la vie courante et rédiger et justifier une démarche. Ces compétences te seront directement utiles pour organiser tes propres fichiers (photos, devoirs, musique) et pour comprendre comment les commerçants, les écoles et les administrations camerounaises gèrent leurs données au quotidien.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 ORGANISATION DES DONNÉES EN INFORMATIQUE

10.1 Méthodes et exercices résolus

MÉTHODE Différencier donnée, information et identifier le type de donnée

Objectif : Savoir distinguer la donnée brute de l'information traitée et classer une donnée selon sa nature (type).

1. Identifier la source : Observer le fait brut, non interprété (chiffre, mesure, symbole). C'est la **donnée**.
2. Analyser le traitement : Repérer l'opération qui donne du sens (contexte, structure, calcul). Le résultat est l'**information**.
3. Classifier le type :
 - **Numérique** : Entier (ex : âge, nombre d'élèves) ou Réel (ex : note, température, prix).
 - **Textuelle (Chaîne de caractères)** : Suite de symboles (ex : nom, adresse, matricule).
 - **Booléenne** : Deux valeurs seulement (Vrai/Faux, Oui/Non, 1/0) (ex : admis/recalé, abonné/non-abonné).
 - **Date/Heure** : Repère temporel structuré.
4. Justifier : Écrire une phrase du type : « C'est une donnée de type entier car elle représente un comptage exact sans virgule. »

EXERCICE RÉSOLU 9 Analyse d'une fiche d'inscription au BEPC

Un établissement scolaire enregistre pour chaque candidat : le Matricule (ex : 2025CE00123), le Nom, la Date de naissance, le Sexe (M/F), la Moyenne_3ème (ex : 12,50), et le Statut_Bourse (Oui/Non).

1. Pour chaque rubrique, précisez s'il s'agit d'une donnée ou d'une information au moment de la saisie.
2. Indiquez le type informatique le plus adapté pour stocker chaque rubrique.
3. Le secrétariat calcule le Pourcentage_Reussite de l'établissement. Est-ce une donnée

ou une information? Justifiez.

SOLUTION

Solution détaillée

1. Au moment de la saisie, toutes les rubriques (Matricule, Nom, Date de naissance, Sexe, Moyenne_3ème, Statut_Bourse) sont des données brutes. Elles deviennent des informations seulement après traitement (ex : tri par moyenne, édition de la liste des boursiers, statistiques par sexe).

2. Types adaptés :

Rubrique	Type informatique	Justification
Matricule	Chaîne de caractères (String)	Contient des lettres et des chiffres, pas de calcul mathématique.
Nom	Chaîne de caractères (String)	Texte pur.
Date de naissance	Date (Date-Time)	Structure Jour/Mois/Année, permet calcul d'âge.
Sexe	Caractère (Char) ou Booléen	Valeurs figées « M » / « F ». Booléen possible (Homme=Vrai).
Moyenne_3ème	Réel (Float/-Double)	Nombre à virgule (note sur 20).
Statut_Bourse	Booléen	Deux états exclusifs : Oui (Vrai) / Non (Faux).

3. Le `Pourcentage_Reussite` est une information. Il résulte d'un traitement (calcul :

$\frac{\text{Nb admis}}{\text{Nb présents}} \times 100$) sur l'ensemble des données brutes (les notes de chaque élève). Il a un sens global pour la prise de décision (évaluation de l'établissement).

Conclusion : La distinction se fait par la présence ou non d'un traitement donnant du sens ; le choix du type dépend de la nature des valeurs et des opérations prévues (calcul, tri, affichage).

MÉTHODE Organiser des données de façon structurée (Fichier, Enregistrement, Champ)

Objectif : Modéliser un ensemble de données cohérent en identifiant les entités, les attributs (champs) et la clé primaire.

1. Identifier l'entité : Quel objet du monde réel gère-t-on ? (Ex : Élève, Livre, Produit). Cela donne le nom du **fichier** (ou table).
2. Lister les attributs (champs) : Quelles caractéristiques faut-il retenir pour chaque occurrence ? Définir le nom, le type et la taille max pour chaque champ.
3. Choisir la clé primaire : Repérer l'attribut (ou groupe d'attributs) unique et non nul pour chaque occurrence (ex : Matricule, ISBN, Code_Produit). C'est l'identifiant unique (**Clé primaire**).
4. Représenter la structure : Sous forme de dictionnaire des données ou schéma : NomFichier (CléPrimaire, Champ2, Champ3, ...).
5. Respecter l'atomicité : Un champ ne doit contenir qu'une seule valeur élémentaire (pas de « Nom et Prénom » dans un seul champ, pas de liste de notes dans un champ).

EXERCICE RÉSOLU 10 Modélisation d'une bibliothèque scolaire

Le bibliothécaire du Lycée de Biyem-Assi veut informatiser la gestion des ouvrages. Pour chaque livre, on note : un code unique (ex : LIV-001), le titre, l'auteur, l'année d'édition, la catégorie (Roman, Sciences, Histoire), le nombre d'exemplaires disponibles.

1. Proposez une structure de fichier adaptée (nom, champs, types, clé primaire).
2. Donnez un exemple d'enregistrement (une ligne) pour l'ouvrage « L'Enfant noir » de Camara Laye, édité en 1953, catégorie Roman, 3 exemplaires, code LIV-045.
3. Pourquoi le « Titre » ne peut-il pas servir de clé primaire ?

SOLUTION

Solution détaillée

1. Structure du fichier : LIVRE (Code_Livre, Titre, Auteur, Annee_Edition,

Categorie, Nb_Exemplaires)

- Code_Livre : Chaîne (Clé Primaire) — Identifiant unique.
- Titre : Chaîne (ex : 100 caractères).
- Auteur : Chaîne (ex : 50 caractères).
- Annee_Edition : Entier (4 chiffres).
- Categorie : Chaîne (ex : 20 caractères) ou Énumération.
- Nb_Exemplaires : Entier (positif ou nul).

2. Enregistrement exemple : LIV-045 | L'Enfant noir | Camara Laye | 1953 | Roman | 3

3. Le Titre n'est pas une clé primaire car :

- Non-unicité : Plusieurs livres différents peuvent avoir le même titre (ex : « Anthologie de la poésie camerounaise » tome 1 et tome 2, ou rééditions).
- Instabilité : Le titre peut changer (nouvelle édition, correction de coquille), une clé primaire ne doit jamais changer.
- Non-atOMICité potentielle : Titre + Sous-titre peut être long et variable.

Seule Code_Livre garantit l'unicité absolue et la stabilité.

Conclusion : Une bonne organisation structurelle repose sur l'atomicité des champs et le choix d'une clé primaire technique, stable et unique.

MÉTHODE Convertir un entier décimal positif en binaire (et inversement)

Objectif : Maîtriser la codification des nombres entiers en base 2 (langage machine).

1. Décimal → Binaire (Division successive par 2) :

- Diviser le nombre N par 2. Noter le quotient Q et le reste R (0 ou 1).
- Recommencer avec Q tant que $Q \neq 0$.
- Le binaire se lit de bas en haut (du dernier reste au premier) : $(R_{\text{dernier}} \dots R_1 R_0)_2$.

2. Binaire → Décimal (Puissances de 2) :

- Numéroter les bits de droite à gauche en partant de 0 (poids $2^0, 2^1, 2^2 \dots$).
- Multiplier chaque bit par 2^{position} .
- Faire la somme : $\sum \text{bit}_i \times 2^i$.

3. Vérification rapide : Un nombre pair finit par 0 en binaire, impair par 1. 2^n s'écrit 1 suivi de n zéros.

EXERCICE RÉSOLU 11 Codage binaire des effectifs d'une classe

Une classe de 3ème compte 47 élèves. Le proviseur veut connaître le codage binaire de cet effectif pour configurer un tableau d'affichage numérique à 8 segments (1 octet).

1. Convertir l'entier 47 en binaire sur 8 bits (1 octet).
2. L'affichage reçoit le code binaire 00101101. Quel effectif cela représente-t-il?
3. Combien d'élèves au maximum peut-on afficher avec 8 bits? Justifiez.

SOLUTION

Solution détaillée

1. 47 (décimal) → Binaire (Division par 2) :

Division	Quotient	Reste
47/2	23	1 (LSB - bit de poids faible)
23/2	11	1
11/2	5	1
5/2	2	1
2/2	1	0
1/2	0	1 (MSB - bit de poids fort)

Lecture des restes de bas en haut : 101111. Sur 8 bits (complété par des zéros à gauche) : 00101111.

2. Binaire 00101101 → Décimal :

$$\begin{aligned} & 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ & = 0 + 0 + 32 + 0 + 8 + 4 + 0 + 1 \\ & = 45 \end{aligned}$$

L'effectif affiché est 45 élèves.

3. Maximum sur 8 bits : $2^8 - 1 = 256 - 1 = 255$. Justification : Avec n bits, on peut coder 2^n valeurs distinctes (de 0 à $2^n - 1$). Pour $n = 8$, valeurs de 0 à 255. 47 et 45 sont bien dans cet intervalle.

Conclusion : La méthode des divisions successives (euclidiennes) est systématique pour l'encodage ; la somme des puissances de 2 pour le décodage. Attention à l'ordre de lecture des restes (LSB vers MSB).

MÉTHODE

Calculer la taille de stockage d'un fichier texte ou image

Objectif : Estimer l'espace mémoire nécessaire (octets, Kio, Mio, Gio) en fonction du codage utilisé.

1. Texte (Codage caractères) :

- **ASCII / ISO-8859-1 / Windows-1252** : 1 octet (8 bits) par caractère → Taille = Nb_caractères × 1 octet.
- **UTF-8** : Variable. 1 octet pour ASCII standard (A-Z, 0-9), 2 octets pour accents (é, è, à), 3 octets pour sinogrammes/arabes, 4 pour émojis. Approximation courante en 3ème : 1 à 2 octets/caractère pour du français.
- **UTF-16** : 2 octets fixes (parfois 4) par caractère.

2. Image (Bitmap brut non compressé) :

- Résolution : $L \times H$ pixels (Largeur × Hauteur).
- Profondeur (Couleurs) : Nb bits par pixel (ex : 1 bit = N&B, 8 bits = 256 couleurs/niveaux de gris, 24 bits = Couleurs vraies / **True Color**).
- Taille brute (bits) = $L \times H \times \text{Profondeur}$.
- Taille (Octets) = $\frac{\text{Taille (bits)}}{8}$.

3. Conversion d'unités (Puissances de 2 - Norme CEI/IEC) :

1 Kio (Kibiocet) = 2^{10} = 1 024 octets

1 Mio (Mébiocet) = 2^{20} = 1 048 576 octets

1 Gio (Gibiocet) = 2^{30} octets

Note : Les constructeurs de disques durs utilisent souvent les puissances de 10 (Ko = 1000 o). Au BEPC, on utilise les puissances de 2 (Kio/Mio) pour la mémoire vive et les tailles de fichiers OS.

EXERCICE RÉSOLU 12

Taille d'un fichier liste d'élèves et d'une photo de classe

1. Un fichier texte `liste_3eme.txt` contient 120 lignes. Chaque ligne : « Matricule Nom Prénom Moyenne » (en moyenne 35 caractères par ligne, accents compris). Le fichier est encodé en UTF-8. Estimer la taille en Kio.
2. La photo de classe fait 3000 × 2000 pixels, en couleur vraie (24 bits/pixel), non compressée (format BMP brut). Calculer sa taille exacte en Mio.

SOLUTION

Solution détaillée

1. Fichier texte (UTF-8) :

- Nombre total de caractères $\approx 120 \times 35 = 4\,200$ caractères.
- En UTF-8, les caractères ASCII (chiffres, lettres sans accent, espaces, ponctuation) = 1 octet. Les accents (é, è, ê, à, ô...) = 2 octets.
- Estimation prudente : $\approx 10\%$ d'accents $\rightarrow 420 \times 2 + 3780 \times 1 = 4\,620$ octets. Arrondissons à 4 700 octets.
- Conversion en Kio : $\frac{4700}{1024} \approx 4,59$ Kio.

2. Image BMP (24 bits/pixel) :

$$\text{Nb pixels} = 3\,000 \times 2\,000 = 6\,000\,000 \text{ pixels}$$

$$\text{Taille (bits)} = 6\,000\,000 \times 24 = 144\,000\,000 \text{ bits}$$

$$\text{Taille (octets)} = \frac{144\,000\,000}{8} = 18\,000\,000 \text{ octets}$$

$$\text{Taille (Mio)} = \frac{18\,000\,000}{1\,048\,576} \approx 17,17 \text{ Mio}$$

(Souvent arrondi à 17,2 Mio ou 18 Mo si puissance de 10).

Conclusion : Le texte est léger (quelques Kio). L'image brute non compressée est lourde (dizaines de Mio), d'où l'intérêt de la compression (JPG, PNG) pour le stockage et le transfert (ex : envoi par WhatsApp ou email au Cameroun).

MÉTHODE Comprendre le codage des caractères : ASCII, Unicode (UTF-8)

Objectif : Expliquer pourquoi l'ASCII est insuffisant et comment l'Unicode (UTF-8) résout le problème des langues du monde.

1. ASCII (7 bits $\rightarrow$ 128 codes) : Historiquement standard US. Codes 0–31 (contrôle), 32–127 (imprimables : A–Z, a–z, 0–9, symboles). Problème : Pas d'accents (é, ç, à), pas de caractères africains (ñ, Ë, Ì), pas d'arabe, chinois, émojis.
2. ISO-8859-1 (Latin-1, 8 bits = 256 codes) : Étend l'ASCII pour l'Europe de l'Ouest (ajoute é, è, à, ç, ñ...). Problème : Incompatible entre zones géographiques (Latin-2 pour l'Est, Latin-6 pour l'Arabe...). Un fichier lu avec le mauvais encodage affiche des « » (mojibake).
3. Unicode (Standard universel) : Attribue un point de code unique (entier) à chaque caractère de toutes les écritures (U+0000 à U+10FFFF). Ex : 'A'=U+0041, 'é'=U+00E9, 'ñ'=U+00F1, '€'=U+20AC.
4. UTF-8 (Encodage Unicode dominant sur le Web/OS) : Codage à longueur variable (1 à 4 octets).
 - 1 octet (0xxxxxxx) : ASCII pur (compatibilité totale ascendante).
 - 2 octets (110xxxxx 10xxxxxx) : Alphabets étendus (Latin, Grec, Cyrillique, Arabe,

Hébreu, Caractères africains spéciaux).

- 3 octets : Sinogrammes (Chinois, Japonais, Coréen – BMP).
- 4 octets : Caractères rares, Émojis, Écritures historiques.

5. Règle d'or : Toujours déclarer/sauvegarder en UTF-8 (éditeur de code, base de données, page HTML `<meta charset="UTF-8">`).

EXERCICE RÉSOLU 13 Diagnostic d'un problème d'affichage « Mojibake »

Un élève de 3ème au Lycée de Ngaoundéré ouvre un fichier `notes.csv` reçu par email. Il voit : Nom : KOUAMÿ ; Note : 14/20 au lieu de KOUAMÉ.

1. Expliquez l'origine technique de ce défaut d'affichage.
2. Le fichier a été créé sur un vieux système Windows (encodage par défaut Windows-1252 / ANSI) et lu sur un système moderne (Linux/Windows 10/11) qui attend de l'UTF-8. Donnez le code hexadécimal UTF-8 correct pour le caractère « É » (U+00C9).
3. Proposez deux solutions concrètes pour l'élève afin de lire correctement son fichier.

SOLUTION

Solution détaillée

1. Origine : C'est un classique conflit d'encodage (Mojibake). Le fichier a été encodé en Windows-1252 (1 octet par caractère étendu). Le caractère « É » y a pour code hexadécimal C9 (1 octet). Le lecteur moderne (UTF-8) lit l'octet C9 comme le début d'une séquence sur 2 octets (car C9 = 11001001_2 commence par 110). Il attend un second octet de continuation (10xxxxxx) qui n'existe pas (ou est le caractère suivant « ; »). Ne trouvant pas de séquence valide, il affiche le caractère de remplacement (U+FFFD) ou « ÿ » selon la police.
2. Code UTF-8 de « É » (U+00C9) : U+00C9 est dans l'intervalle U+0080..U+07FF → Codage sur 2 octets. Binaire du point de code : 000 1100 1001 (11 bits utiles). Répartition UTF-8 : 110xxxxx 10xxxxxx → 11000011 10001001 → Hexadécimal : C3 89. Vérification : C3 = 11000011 (début 2 octets), 89 = 10001001 (continuation). Correct.
3. Solutions pour l'élève :
 - (a) Ouvrir avec un éditeur avancé (Notepad++, VS Code, Sublime Text) : Menu Encodage → Character Sets → Western European → Windows-1252. Le texte s'affiche bien. Puis Encodage → Convertir en UTF-8 et Sauvegarder.
 - (b) En ligne de commande (Linux/Termux/WSL) : `iconv -f WINDOWS-1252 -t UTF-8 notes.csv > notes_utf8.csv.`

Conclusion : L'UTF-8 est le standard universel. Tout fichier texte échangé (email, web, clé USB) doit être en UTF-8 pour éviter ce problème, surtout dans un pays multilingue comme

le Cameroun (français, anglais, langues nationales).

MÉTHODE Passer de l'algorithme au code C : Structure de base, Entrées/Sorties, Types

Objectif : Écrire un programme C complet, correct, compilable (GCC), respectant la syntaxe stricte pour résoudre un problème simple (calcul, conversion).

1. Inclusions : `#include <stdio.h>` (indispensable pour `printf`, `scanf`).
2. Fonction principale : `int main(void) { ... return 0; }`.
3. Déclaration des variables : Au début du bloc (C90) ou près de l'usage (C99). Types de base : `int` (entier), `float` / `double` (réel), `char` (caractère/octet).
4. Affichage (Sortie) : `printf("Format", variables);`
 - `%d` ou `%i` pour `int`.
 - `%f` pour `float` (par défaut 6 décimales), `%.2f` pour 2 décimales.
 - `%c` pour `char`, `%s` pour chaîne.
 - `\n` pour retour à la ligne.
5. Saisie (Entrée) : `scanf("Format", &variable);`
 - Obligatoire : Le `&` (esperluette / adresse de) devant le nom de la variable (sauf pour les tableaux/chaînes).
 - Le format doit correspondre exactement au type.
6. Opérateurs arithmétiques : `+` `-` `*` `/` `%` (modulo, reste entier). Attention : `/` sur deux `int` fait une division entière (tronquée). Forcer le réel : `(float)a / b` ou `a / 2.0`.
7. Compilation / Exécution (Terminal) : `gcc -Wall -o prog prog.c puis ./prog`.

EXERCICE RÉSOLU 14 Programme C : Conversion Binaire → Décimal

Écrire un programme C complet qui :

- Demande à l'utilisateur de saisir un entier positif représentant un nombre binaire (ex : 101111).
- Vérifie que la saisie ne contient que des 0 et des 1 (optionnel mais recommandé).
- Calcule et affiche l'équivalent décimal.

Indication : Traiter le nombre saisi comme un entier (type `long long`) et extraire les chiffres par modulo 10 et division par 10.

SOLUTION

Solution détaillée (Code C commenté)

```
#include <stdio.h>
#include <stdbool.h> // Pour le type bool, true, false

int main(void) {
    long long binaire; // Type assez grand pour stocker jusqu'à 18-19 chiffres binaires (64 bits)
    int decimal = 0;
    int poids = 1; // 2^0 = 1 au départ
    bool valide = true;

    // 1. Saisie
    printf("Entrez un nombre binaire (ex : 101111) : ");
    // On lit comme un entier. Si l'utilisateur tape 102, binaire vaudra 102.
    if (scanf("%lld", &binaire) != 1) {
        printf("Erreur de saisie.\n");
        return 1;
    }

    // 2. Validation et Conversion simultanées
    long long temp = binaire; // Copie pour ne pas perdre la valeur originale si besoin

    if (temp == 0) {
        decimal = 0; // Cas particulier : 0 binaire = 0 decimal
    } else {
        while (temp > 0) {
            int bit = temp % 10; // Extrait le chiffre de droite (poids faible)

            // Vérification : un chiffre binaire ne peut être que 0 ou 1
            if (bit != 0 && bit != 1) {
                valide = false;
                break; // Sort de la boucle immédiatement
            }

            // Calcul décimal : bit * 2^position
            // Ici 'poids' vaut 1, 2, 4, 8, 16... (puissances de 2 successives)
            decimal += bit * poids;

            // Préparation itération suivante
            poids *= 2; // Poids suivant (x2)
            temp /= 10; // Décale le nombre vers la droite (supprime le chiffre traité)
        }
    }

    // 3. Affichage résultat
    if (valide) {
        printf("Binaire saisi : %lld\n", binaire);
        printf("Equivalent decimal : %d\n", decimal);
    } else {
        printf("Erreur : '%lld' n'est pas un nombre binaire valide (que des 0 et 1).\n", binaire);
    }

    return 0; // Succès
}
```

-

-

-

-

-

MÉTHODE Résoudre un problème concret par l'algorithmique (Analyse, Algo, Codage, Test)

Objectif : Appliquer la démarche complète de résolution informatique à une situation de la vie courante (gestion de notes, stock, budget).

1. Analyser la situation : Identifier les données d'entrée (connues, saisies), les résultats attendus (sorties), et les contraintes/règles de gestion (formules, seuils, arrondis).
2. Concevoir l'algorithme (Pseudo-code) :
 - Variables (Noms clairs, Types).
 - Séquence logique : Début → Saisir → Traiter (Si/Pour/TantQue) → Afficher → Fin.
 - Indentation rigoureuse (2 ou 4 espaces) pour voir les blocs.
3. Coder en C (ou Python/JS si requis) : Transcrire fidèlement l'algorithme. Respecter la syntaxe (; , { } , & pour scanf).
4. Tester (Jeu de tests) :
 - Cas nominal (valeurs normales).
 - Cas limites (0, valeur max, seuil exact).
 - Cas erronés/invalides (négatif, texte au lieu de nombre, division par 0).
5. Documenter / Conclure : Commenter le code (// but de la variable, // formule). Rédiger la phrase de réponse finale.

EXERCICE RÉSOLU 15 Gestion du stock d'un cybercafé (Algorithme + C)

Le gérant du cybercafé « @BiyemAssi » veut un programme pour gérer les recharges de temps. Règles :

- 1 heure = 300 FCFA.
 - Si le client achète ≥ 5 heures, il bénéficie de 10% de remise sur le total.
 - Le programme demande le nombre d'heures souhaité (entier > 0), calcule et affiche le montant à payer (entier FCFA, arrondi au plus bas).
1. Écrire l'algorithme en pseudo-code français structuré.
 2. Écrire le programme C correspondant.
 3. Donner les résultats pour 3h, 5h, 10h.

SOLUTION

Solution détaillée

1. Algorithme (Pseudo-code) :

```
Algorithme CalculRechargeCyber
Variables
    nbHeures : Entier
    tarifHoraire, total, remise, aPayer : Reel (* ou Entier pour FCFA *)
Debut
    Afficher "Bienvenue au Cyber @BiyemAssi"
    Afficher "Tarif : 300 FCFA/h. Remise 10% si  $\geq 5$ h."
    Repeter
        Afficher "Nombre d'heures desirées (entier  $> 0$ ) : "
        Saisir nbHeures
    Jusqu'A nbHeures  $> 0$ 

    tarifHoraire  $\leftarrow 300$ 
    total  $\leftarrow$  nbHeures * tarifHoraire

    Si nbHeures  $\geq 5$  Alors
        remise  $\leftarrow$  total * 0.10
    Sinon
        remise  $\leftarrow 0$ 
    FinSi

    aPayer  $\leftarrow$  total - remise
    // Arrondi au plus bas (entier FCFA) : conversion implicite Entier ou floor()
    Afficher "Total brut : ", total, " FCFA"
    Afficher "Remise : ", remise, " FCFA"
    Afficher "Montant à payer : ", PartieEntiere(aPayer), " FCFA"
Fin
```

2. Programme C :

```
#include <stdio.h>
```

```

int main(void){
    int nbHeures;
    const int TARIF = 300; // Constante pour le tarif
    int total, remise, aPayer; // Tout en entiers (FCFA) pour éviter erreurs float

    printf("=== Cyber @BiyemAssi ===\n");
    printf("Tarif : %d FCFA/h. Remise 10%% si >= 5h.\n", TARIF);

    // Saisie sécurisée (boucle tant que invalide)
    do {
        printf("Nombre d'heures desirées (entier > 0) : ");
        if (scanf("%d", &nbHeures) != 1) { // Gestion saisie non-numérique
            printf("Erreur : saisir un nombre entier.\n");
            while(getchar() != '\n'); // Vider le buffer clavier
            nbHeures = 0; // Forcer reprise boucle
        }
    } while (nbHeures <= 0);

    total = nbHeures * TARIF;

    // Calcul remise en entier : (total * 10) / 100 = total / 10
    // Division entière = arrondi au plus bas automatique en C pour positifs
    if (nbHeures >= 5) {
        remise = total / 10;
    } else {
        remise = 0;
    }

    aPayer = total - remise;

    printf("-----\n");
    printf("Total brut : %d FCFA\n", total);
    printf("Remise (10%%) : %d FCFA\n", remise);
    printf("A PAYER : %d FCFA\n", aPayer);

    return 0;
}

```

3. Jeu de tests (Vérification manuelle) :

Heures	Total Brut	Condition	Remise (Total/10)	À Payer
3 h	3 × 300 = 900	< 5	0	900 FCFA
5 h	5 × 300 = 1500	≥ 5	1500/10 = 150	1350 FCFA
10 h	10 × 300 = 3000	≥ 5	3000/10 = 300	2700 FCFA

Le programme C produit exactement ces résultats. L'utilisation d'entiers pour l'argent (FCFA) évite les erreurs d'arrondi du type `float` (ex : 1349.9999 au lieu de 1350).

Conclusion : La démarche structurée (Analyse → Algo → Code → Test) garantit un programme fiable. L'usage de `const` pour les constantes métier (TARIF) et d'entiers pour la monnaie sont des bonnes pratiques professionnelles.

ATTENTION Les 5 erreurs qui coûtent des points au BEPC

1. Confondre Donnée et Information : Dire « La note 12 est une information » sans préciser le contexte. Correction : « La note 12 est une donnée brute. La décision "Admis" prise à partir de la moyenne est une information. »
2. Oublier le & dans scanf : Écrire `scanf("%d", age);` au lieu de `scanf("%d", &age);`. Conséquence : Crash immédiat (Segmentation fault) ou écriture mémoire aléatoire. Règle : & obligatoire pour `int`, `float`, `double`, `char` (variables scalaires).
3. Division entière non voulue : Écrire `moyenne = (note1 + note2) / 2;` avec des `int`. Résultat : 12 au lieu de 12,5. Correction : `moyenne = (note1 + note2) / 2.0;` ou caster (float) `(note1+note2)/2.`
4. Mauvaise lecture des restes (Binaire) : Lire les restes de haut en bas (premier reste = bit de poids fort). Règle : Le premier reste est le bit de poids faible (LSB, bit 0). On lit de bas en haut.
5. Unités de stockage mélangées (Puissances 10 vs 2) : Calculer 1 Mo = 1 000 000 o pour une taille de fichier RAM/Disque OS. Au BEPC Informatique : 1 Kio = 1024 o, 1 Mio = 1024 Kio. Utiliser les puissances de 2 (2^{10} , 2^{20}). Noter l'unité Kio/Mio/Gio (kibi, mebi, gibi) pour être précis.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 8 QCM : à toi de choisir

Pour chaque question, une seule réponse est exacte. Recopie la lettre de la bonne réponse sur ta feuille.

1. Une **donnée brute** est :
 - (a) une information déjà interprétée ;
 - (b) un élément brut non encore traité (chiffre, mot, image) ;
 - (c) un programme de l'ordinateur.
2. L'unité de base de l'information dans l'ordinateur est :
 - (a) le caractère ;
 - (b) le bit ;
 - (c) le kilo-octet.
3. Un octet est formé de :
 - (a) 2 bits ;
 - (b) 8 bits ;
 - (c) 16 bits.
4. Le code **ASCII** sert à représenter :
 - (a) les caractères (lettres, chiffres, symboles) ;
 - (b) uniquement les nombres entiers ;
 - (c) les images.
5. 1 Ko (kilo-octet) vaut environ :
 - (a) 8 octets ;
 - (b) 100 octets ;
 - (c) 1024 octets.

EXERCICE 9 Vrai ou faux ? Justifie

Réponds par Vrai ou Faux, puis justifie ta réponse en une ou deux phrases.

1. Une photo numérique est une information, jamais une donnée.
2. Dans l'ordinateur, toutes les données sont représentées à l'aide des deux chiffres 0 et 1.
3. Le nombre binaire 1011 contient quatre bits.
4. Le code ASCII d'une lettre majuscule est le même que celui de la même lettre en minuscule.
5. Une clé USB de 1 Go peut contenir plus de données qu'une disquette de 1 Mo.

EXERCICE 10 Définitions à compléter

Recopie et complète les phrases suivantes avec les mots : bit, information, donnée, octet, arborescence, binaire.

1. Un élément brut comme la note 14 est une ... ; la phrase « l'élève a la moyenne » est une ...
2. Le ... est la plus petite unité d'information ; il ne peut prendre que les valeurs 0 ou 1.
3. Huit bits forment un ...
4. Le système de numération qui n'utilise que les chiffres 0 et 1 est le système ...
5. L'organisation des dossiers et fichiers sous forme d'arbre s'appelle une ...

EXERCICE 11 Calculs directs

1. Convertis en décimal les nombres binaires suivants en détaillant les puissances de 2 : 101 ; 1100.
2. Convertis en binaire les nombres décimaux suivants par divisions successives par 2 : 10 ; 19.
3. Combien d'octets y a-t-il dans 3 Ko ? dans 2 Mo ?

11.2 Je m'entraîne

EXERCICE 12 Données brutes ou informations ?

Voici une liste d'éléments relevés dans le secrétariat d'un collège : une note de 12/20 ;

la moyenne générale d'un élève ; une photo d'identité ; une chanson enregistrée ; un numéro de téléphone ; la mention « Admis » sur un relevé.

1. Classe chaque élément dans la catégorie donnée brute ou information.
2. Justifie chacun de tes choix en une phrase.

EXERCICE 13 Codage ASCII du mot « ECOLE »

On donne l'extrait suivant de la table ASCII (codes décimaux) :

Caractère	A	B	C	E	L	N	O	R
Code ASCII	65	66	67	69	76	78	79	82

1. Écris le code ASCII décimal de chaque lettre du mot « ECOLE ».
2. Convertis en binaire le code de la lettre « E » et celui de la lettre « O ».
3. À l'aide de la table, décode le message suivant : 66 - 79 - 78 - 74 - 79 - 85 - 82 (le code de J est 74 et celui de U est 85).

EXERCICE 14 La clé USB de Marlyse

Marlyse veut copier sur sa clé USB de 1 Go : 300 photos de 2 Mo chacune et 10 chansons de 5 Mo chacune. On rappelle que 1 Go = 1024 Mo.

1. Calcule l'espace occupé par les photos, puis par les chansons.
2. Calcule l'espace total nécessaire en Mo.
3. La copie est-elle possible sur la clé ? Justifie ta réponse par un calcul et une conclusion rédigée.

EXERCICE 15 L'arborescence du lycée

Le lycée de Nkongsamba organise ses fichiers ainsi : un dossier principal « LYCEE » contient un dossier par niveau (« 2nde », « 1ere », « Tle ») ; chaque niveau contient un dossier par classe ; chaque classe contient un dossier par matière et un dossier « BULLETINS ».

1. Dessine l'arborescence complète pour les classes Tle C et Tle D, avec les matières Maths et Informatique.
2. Écris le chemin d'accès complet du fichier « bulletin1.pdf » de la classe de Tle C, en

commençant par « LYCEE ».

3. Cite deux avantages d'une telle organisation structurée des données.

11.3 Je me prépare au Probatoire

EXERCICE 16 Type BEPC : conversions binaire-décimal

Partie A. Un professeur d'informatique écrit au tableau les nombres 25 et 47.

1. Convertis 25 en binaire par la méthode des divisions successives par 2, en présentant toutes les étapes.
2. Convertis 47 en binaire par la même méthode.
3. Vérifie tes résultats en reconvertissant chaque nombre binaire obtenu en décimal, à l'aide des puissances de 2.

Partie B. Un élève a trouvé sur une affiche le nombre binaire 10110.

1. Convertis 10110 en décimal en détaillant la méthode des puissances de 2.
2. Combien de bits composent ce nombre ? Quel est le bit de poids fort ? le bit de poids faible ?
3. Rédige une conclusion expliquant pourquoi l'ordinateur utilise le binaire plutôt que le décimal.

EXERCICE 17 Type BEPC : le cybercafé de Bafoussam

M. Kamdem gère un cybercafé à Bafoussam. Il possède environ 2000 fichiers clients : des documents Word, des photos scannées et des fichiers de musique, actuellement mélangés dans un seul dossier. Chaque fichier occupe en moyenne 3 Mo.

1. Explique la différence entre une donnée et une information, en donnant un exemple de chaque tiré de la situation de M. Kamdem.
2. Propose une organisation structurée de ses fichiers sous forme d'arborescence (dossiers et sous-dossiers) et dessine-la.
3. Écris le chemin d'accès d'un fichier « contrat.pdf » rangé dans le sous-dossier « Documents » de l'année 2026.
4. Calcule l'espace disque total nécessaire en Mo, puis convertis-le en Go (on rappelle : 1 Go = 1024 Mo).
5. M. Kamdem hésite entre un disque dur de 4 Go et un disque de 10 Go. Lequel conseilles-tu ? Rédige et justifie ta démarche complète.

EXERCICE 18 Type BEPC : le bulletin de la 3e A

Le principal d'un collège de Yaoundé veut numériser les bulletins de la classe de 3e A, qui compte 60 élèves. Chaque bulletin scanné est une image de 500 Ko. On donne l'extrait de table ASCII : A = 65, B = 66, C = 67, E = 69.

1. Le nom du collège commence par le mot « CEB ». Donne le code ASCII décimal de chaque lettre de ce mot, puis convertis le code de la lettre « B » en binaire.
2. Calcule l'espace total occupé par les 60 bulletins en Ko, puis convertis le résultat en Mo (on rappelle : 1 Mo = 1024 Ko).
3. Le principal veut graver ces bulletins sur un CD de 700 Mo. Est-ce possible ? Justifie par un calcul.
4. Propose une arborescence permettant de ranger les bulletins par trimestre (T1, T2, T3) et écris le chemin d'accès du bulletin de l'élève « ABENA » pour le trimestre T1.
5. Explique en quelques lignes comment l'ordinateur représente une image scannée à l'aide de 0 et de 1.

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : à toi de choisir

1. b. Une donnée brute est un élément brut non encore traité (chiffre, mot, image). C'est le traitement qui la transforme en information.
2. b. L'unité de base de l'information est le **bit** (0 ou 1).
3. b. Un octet est formé de 8 bits.
4. a. Le code ASCII sert à représenter les caractères : lettres, chiffres et symboles.
5. c. 1 Ko vaut environ 1024 octets (et non 1000).

CORRIGÉ Exercice 2 — Vrai ou faux ? Justifie

1. Faux. Une photo numérique est d'abord une donnée (un fichier d'image stocké dans l'ordinateur) ; elle devient une information lorsqu'elle est interprétée, par exemple pour identifier un élève.
2. Vrai. Dans l'ordinateur, toutes les données (textes, images, sons, nombres) sont codées en **binaire**, c'est-à-dire à l'aide des deux chiffres 0 et 1, car les circuits électroniques ne connaissent que deux états (courant qui passe ou ne passe pas).
3. Vrai. Le nombre 1011 est écrit avec quatre chiffres binaires ; il contient donc 4 bits.
4. Faux. En ASCII, chaque caractère a un code différent : par exemple « A » vaut 65 et « a » vaut 97. Majuscules et minuscules sont des caractères distincts.
5. Vrai. 1 Go = 1024 Mo, donc 1 Go > 1 Mo : la clé USB contient beaucoup plus de données que la disquette (environ 1024 fois plus).

CORRIGÉ Exercice 3 — Définitions à compléter

1. Un élément brut comme la note 14 est une donnée ; la phrase « l'élève a la moyenne » est une information.
2. Le bit est la plus petite unité d'information ; il ne peut prendre que les valeurs 0 ou 1.
3. Huit bits forment un octet.
4. Le système de numération qui n'utilise que les chiffres 0 et 1 est le système binaire.

5. L'organisation des dossiers et fichiers sous forme d'arbre s'appelle une arborescence.

CORRIGÉ Exercice 4 — Calculs directs

1. Conversion binaire → décimal :

$$101 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5$$
$$1100 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

2. Conversion décimal → binaire par divisions successives par 2 :

- Pour 10 : $10 \div 2 = 5$ reste 0 ; $5 \div 2 = 2$ reste 1 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1. On lit les restes de bas en haut : $10 = 1010_2$.
- Pour 19 : $19 \div 2 = 9$ reste 1 ; $9 \div 2 = 4$ reste 1 ; $4 \div 2 = 2$ reste 0 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1. Lecture de bas en haut : $19 = 10011_2$.

Vérification : $1010_2 = 8 + 2 = 10$ et $10011_2 = 16 + 2 + 1 = 19$. Correct.

3. Conversions d'unités :

- $3 \text{ Ko} = 3 \times 1024 = 3072$ octets ;
- $2 \text{ Mo} = 2 \times 1024 \times 1024 = 2\,097\,152$ octets.

CORRIGÉ Exercice 5 — Données brutes ou informations ?

1. Classement :

Élément	Catégorie
Une note de 12/20	Donnée brute
La moyenne générale d'un élève	Information
Une photo d'identité	Donnée brute
Une chanson enregistrée	Donnée brute
Un numéro de téléphone	Donnée brute
La mention « Admis » sur un relevé	Information

2. Justifications :

- La note 12/20 est un chiffre brut, non encore interprété : c'est une donnée.
- La moyenne générale est le résultat d'un traitement (calcul à partir de plusieurs notes) : c'est une information.

- La photo d'identité est un fichier image brut : c'est une donnée.
- La chanson enregistrée est un fichier son brut : c'est une donnée.
- Le numéro de téléphone est une suite de chiffres brute : c'est une donnée.
- La mention « Admis » résulte de l'interprétation de la moyenne (décision prise) : c'est une information.

CORRIGÉ Exercice 6 — Codage ASCII du mot « ECOLE »

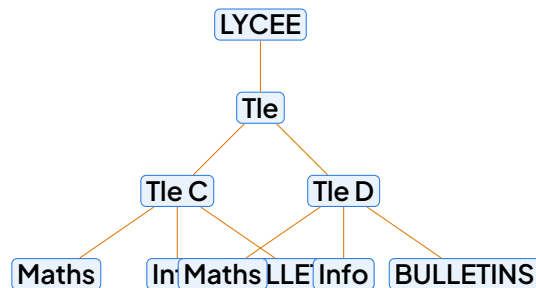
1. D'après la table : E = 69, C = 67, O = 79, L = 76, E = 69.
Le mot « ECOLE » se code donc : 69 - 67 - 79 - 76 - 69.
2. Conversions en binaire (sur 7 bits, code ASCII standard) :
 - « E » = 69 : $69 \div 2 = 34$ reste 1 ; $34 \div 2 = 17$ reste 0 ; $17 \div 2 = 8$ reste 1 ; $8 \div 2 = 4$ reste 0 ; $4 \div 2 = 2$ reste 0 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1. Donc $69 = 1000101_2$.
Vérification : $64 + 4 + 1 = 69$. Correct.
 - « O » = 79 : $79 \div 2 = 39$ reste 1 ; $39 \div 2 = 19$ reste 1 ; $19 \div 2 = 9$ reste 1 ; $9 \div 2 = 4$ reste 1 ; $4 \div 2 = 2$ reste 0 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1. Donc $79 = 1001111_2$.
Vérification : $64 + 8 + 4 + 2 + 1 = 79$. Correct.
3. Décodage : 66 = B ; 79 = O ; 78 = N ; 74 = J ; 79 = O ; 85 = U ; 82 = R.
Le message est : BONJOUR.

CORRIGÉ Exercice 7 — La clé USB de Marlyse

1. Espace occupé :
 - Photos : $300 \times 2 = 600$ Mo ;
 - Chansons : $10 \times 5 = 50$ Mo.
2. Espace total : $600 + 50 = 650$ Mo.
3. La clé a une capacité de 1 Go = 1024 Mo. Or $650 < 1024$: il restera même $1024 - 650 = 374$ Mo libres.
Conclusion : la copie est possible, car l'espace total nécessaire (650 Mo) est inférieur à la capacité de la clé (1024 Mo).

CORRIGÉ Exercice 8 — L'arborescence du lycée

1. Arborescence demandée :



Arborescence des fichiers du lycée (niveau Terminale, classes C et D).

2. Chemin d'accès complet : LYCEE\Tle\Tle C\BULLETINS\bulletin1.pdf

3. Deux avantages d'une organisation structurée :

- on retrouve rapidement un fichier grâce à son chemin d'accès, sans chercher parmi des milliers de fichiers mélangés ;
- on évite les confusions entre fichiers (par exemple les bulletins de deux classes différentes) et on facilite les sauvegardes et le partage.

CORRIGÉ Exercice 9 — Type BEPC : conversions binaire-décimal

Partie A.

1. Conversion de 25 : $25 \div 2 = 12$ reste 1 ; $12 \div 2 = 6$ reste 0 ; $6 \div 2 = 3$ reste 0 ; $3 \div 2 = 1$ reste 1 ; $1 \div 2 = 0$ reste 1.

Lecture des restes de bas en haut : $25 = 11001_2$.

2. Conversion de 47 : $47 \div 2 = 23$ reste 1 ; $23 \div 2 = 11$ reste 1 ; $11 \div 2 = 5$ reste 1 ; $5 \div 2 = 2$ reste 1 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1.

Donc $47 = 101111_2$.

3. Vérifications :

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 1 = 25 \quad \checkmark$$

$$101111_2 = 2^5 + 2^3 + 2^2 + 2^1 + 2^0 = 32 + 8 + 4 + 2 + 1 = 47 \quad \checkmark$$

Partie B.

1. $10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 16 + 0 + 4 + 2 + 0 = 22$.

2. Ce nombre est composé de 5 bits. Le bit de poids fort (le plus à gauche, qui vaut $2^4 = 16$) est 1 ; le bit de poids faible (le plus à droite, qui vaut $2^0 = 1$) est 0.

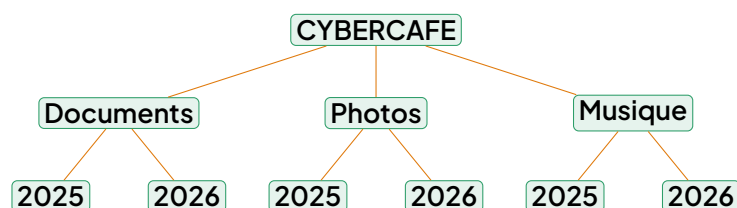
3. Conclusion : l'ordinateur utilise le binaire car ses circuits électroniques ne peuvent

distinguer que deux états physiques stables : le courant passe (1) ou ne passe pas (0). Coder l'information avec deux symboles seulement rend la machine simple, rapide et fiable, alors que représenter dix chiffres exigerait de distinguer dix niveaux de tension différents, source d'erreurs.

Barème indicatif (10 points) : A1 : 2 pts ; A2 : 2 pts ; A3 : 2 pts ; B1 : 1,5 pt ; B2 : 1 pt ; B3 : 1,5 pt. Points de vigilance : présenter toutes les divisions successives (quotient et reste) ; lire les restes de bas en haut ; dans la méthode des puissances de 2, commencer par 2^0 à droite ; citer explicitement les deux états électriques dans la conclusion.

CORRIGÉ Exercice 10 — Type BEPC : le cybercafé de Bafoussam

1. Une donnée est un élément brut non traité : par exemple un fichier photo scanné d'un client. Une information est une donnée interprétée ou traitée qui a un sens : par exemple la phrase « ce client doit encore payer 500 F », obtenue à partir de ses factures.
2. Organisation proposée : un dossier principal « CYBERCAFE » contenant trois sous-dossiers « Documents », « Photos » et « Musique », chacun subdivisé par année.



Arborescence proposée pour les fichiers du cybercafé.

3. Chemin d'accès : CYBERCAFE\Documents\2026\contrat.pdf
4. Espace total : $2000 \times 3 = 6000 \text{ Mo}$.
Conversion : $6000 \div 1024 \approx 5,86 \text{ Go}$.
5. Démarche : l'espace nécessaire est d'environ 5,86 Go. Le disque de 4 Go est insuffisant car $4 < 5,86$. Le disque de 10 Go convient car $10 > 5,86$, et il laisse environ $10 - 5,86 = 4,14 \text{ Go}$ de libre pour les nouveaux fichiers.
Conclusion : je conseille à M. Kamdem le disque de 10 Go, car il peut contenir tous ses fichiers actuels et offre une marge pour l'avenir.

Barème indicatif (10 points) : 1 : 2 pts ; 2 : 2 pts ; 3 : 1 pt ; 4 : 2 pts ; 5 : 3 pts.

Points de vigilance : distinguer clairement donnée/information avec des exemples tirés de la situation ; l'arborescence doit avoir un dossier racine et des sous-dossiers ; le chemin commence par le dossier principal ; la conversion Mo $\rightarrow$ Go se fait par division par 1024 ; la réponse à la question 5 doit comparer les deux disques et conclure par une

phrase rédigée.

CORRIGÉ Exercice 11 — Type BEPC : le bulletin de la 3e A

1. Codes ASCII de « CEB » : C = 67, E = 69, B = 66.
Conversion de « B » = 66 en binaire : $66 \div 2 = 33$ reste 0 ; $33 \div 2 = 16$ reste 1 ; $16 \div 2 = 8$ reste 0 ; $8 \div 2 = 4$ reste 0 ; $4 \div 2 = 2$ reste 0 ; $2 \div 2 = 1$ reste 0 ; $1 \div 2 = 0$ reste 1.
Donc $66 = 1000010_2$. Vérification : $64 + 2 = 66$. Correct.
2. Espace total : $60 \times 500 = 30\,000$ Ko.
Conversion : $30\,000 \div 1024 \approx 29,3$ Mo.
3. Le CD a une capacité de 700 Mo. Or $29,3 < 700$: la gravure est possible, et il restera environ $700 - 29,3 = 670,7$ Mo libres.
4. Arborescence : un dossier « BULLETINS_3eA » contenant trois sous-dossiers « T1 », « T2 » et « T3 ».
Chemin d'accès : BULLETINS_3eA\T1\ABENA.pdf
5. Une image scannée est découpée en petits points appelés **pixels**. À chaque pixel, l'ordinateur associe un nombre qui représente sa couleur (ou son niveau de gris). Chaque un de ces nombres est ensuite converti en binaire, c'est-à-dire en une suite de 0 et de 1. L'image entière est donc stockée comme une longue suite de bits, que l'ordinateur peut reconstituer à l'écran.

Barème indicatif (10 points) : 1 : 2 pts ; 2 : 2 pts ; 3 : 1,5 pt ; 4 : 2 pts ; 5 : 2,5 pts.

Points de vigilance : pour le codage ASCII, donner les trois codes et la conversion binaire complète de « B » ; la conversion Ko → Mo est une division par 1024 ; la question 3 exige une comparaison chiffrée et une conclusion rédigée ; pour la question 5, les mots-clés attendus sont pixels, couleur, nombre, binaire.

13 FICHE BILAN

Carte mentale : Organisation des données en informatique (Leçon N04)

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Donnée** : Représentation brute d'un fait, d'une mesure, d'un symbole (ex. : 23, A, true). Sans contexte, elle n'a pas de sens.
- **Information** : Donnée traitée, contextualisée, qui a un sens pour le destinataire et permet une décision (ex. : « La température est de 23 °C »).
- **Type de données** : Classification qui détermine les valeurs possibles et les opérations autorisées. Principaux types élémentaires : **Entier**, **Réel**, **Booléen**, **Caractère**, **Chaîne de caractères**.
- **Structure de données** : Organisation logique des données en mémoire (tableau, liste,

pile, fichier, arbre, table de base de données).

- **Bit** (Binary Digit) : Plus petite unité d'information, vaut 0 ou 1.
- **Octet** (Byte) : Groupe de 8 bits. Unité de base de l'adressage mémoire. 1 octet = 8 bits.
- **Codage** : Convention de représentation d'une information (nombre, texte, image, son) par une suite de bits.

Organisation structurée des données (Niveaux)

Niveau	Description	Exemple
Bit	Unité élémentaire	0, 1
Octet	8 bits	01000001 (code ASCII 'A')
Champ / Zone	Donnée élémentaire identifiée (nom + type + valeur)	Nom : "KAMGA"
Enregistrement / Tuple	Ensemble de champs liés à un même objet	Fiche élève : {Matricule, Nom, Classe}
Fichier / Table	Ensemble d'enregistrements de même structure	Fichier ELEVES.csv
Base de données	Ensemble de fichiers liés, gérés par un SGBD	Base CollegeDB

Représentation des nombres : Conversions à maîtriser

Conversion	Méthode express
Décimal → Binaire	Divisions successives par 2, lire les restes de bas en haut.
Binaire → Décimal	Somme des puissances de 2 : $\sum b_i \times 2^i$ (poids de droite à gauche : 1, 2, 4, 8 ...).
Binaire ↔ Hexadécimal	Groupes de 4 bits (nibble) ↔ 1 chiffre hexa (0 ... 9, A ... F).
Décimal → Hexadécimal	Via le binaire ou divisions successives par 16.
Entiers signés	Complément à deux : Inverser les bits + ajouter 1. Le bit de poids fort (MSB) indique le signe (0 = +, 1 = -).
Réels (approche)	Notation scientifique binaire (standard IEEE 754) : Signe × Mantisse × 2^{Exposant} .

Codage des caractères

- **ASCII** (7 bits, 128 codes) : Contrôles (0–31), Chiffres (48–57), Majuscules (65–90), Minuscules (97–122). Étendu à 8 bits (256 codes) pour accents (ISO–8859–1).
- **Unicode / UTF–8** : Standard universel. UTF–8 : codage à longueur variable (1 à 4 octets). Compatible ASCII sur 1 octet. Indispensable pour le web et le multilinguisme (dont langues camerounaises).

Unités de mesure (Puissances de 2)

$$\begin{aligned}1 \text{ Kio (Kibi-octet)} &= 2^{10} \text{ octets} = 1\,024 \text{ octets} \\1 \text{ Mio (Mébi-octet)} &= 2^{20} \text{ octets} = 1\,024 \text{ Kio} \\1 \text{ Gio (Gibi-octet)} &= 2^{30} \text{ octets} = 1\,024 \text{ Mio} \\1 \text{ Tio (Tébi-octet)} &= 2^{40} \text{ octets} = 1\,024 \text{ Gio}\end{aligned}$$

Méthodes express

MÉTHODE Convertir un entier décimal positif en binaire

1. Diviser le nombre par 2.
2. Noter le reste (0 ou 1).
3. Recommencer avec le quotient jusqu'à obtenir 0.
4. Le binaire se lit des restes du dernier au premier (bas vers haut).

MÉTHODE Convertir un binaire en décimal

1. Écrire les poids des bits (puissances de 2) au-dessus de chaque bit, en partant de $2^0 = 1$ à droite.
2. Additionner les poids correspondant aux bits à 1.

MÉTHODE Calculer la taille d'un fichier image (bitmap non compressé)

1. Résolution = Largeur × Hauteur (en pixels).
2. Profondeur = Nombre de bits par pixel (ex. 24 bits = 3 octets pour RVB Vrai Couleur).
3. Taille (bits) = Résolution × Profondeur.

4. Taille (octets) = Taille (bits) / 8.

À RETENIR Checklist avant le Bac

- ☐ Différencier donnée (brute) et information (contextualisée).
- ☐ Citer les 5 types de données élémentaires et donner un exemple de valeur pour chacun.
- ☐ Expliquer la hiérarchie : Bit → Octet → Champ → Enregistrement → Fichier → Base de données.
- ☐ Convertir un entier décimal ↔ binaire (ex. $42_{10} = 101010_2$).
- ☐ Convertir binaire ↔ hexadécimal par groupes de 4 bits (ex. $1010_2 = A_{16}$).
- ☐ Représenter un entier négatif en complément à deux sur 8 bits (ex. -5).
- ☐ Expliquer la différence entre ASCII (7/8 bits, limité) et UTF-8 (variable, universel).
- ☐ Calculer la taille en octets d'un fichier texte (1 octet/caractère ASCII, variable UTF-8) ou image brute (Largeur × Hauteur × Profondeur/8).
- ☐ Convertir des unités de stockage : Kio, Mio, Gio, Tio (puissances de 1024) vs ko, Mo, Go (puissances de 1000).
- ☐ Identifier le codage adapté à une situation : nombre entier (binaire/2e complément), réel (IEEE 754), texte (UTF-8), image (RVB + compression), son (échantillonnage).
- ☐ Justifier l'intérêt de l'organisation structurée (accès rapide, intégrité, partage, sécurité).
- ☐ Résoudre un problème concret : « Combien d'octets pour stocker 500 noms de 20 caractères en UTF-8 ? » (Réponse : $\approx 500 \times 20 \times 1 = 10\,000$ octets si ASCII, plus si accents).

ATTENTION Notation commerciale

Les constructeurs de disques durs utilisent souvent les puissances de 10 ($ko = 10^3$, $Mo = 10^6$). L'OS affiche en puissances de 2 (Kio, Mio). D'où la différence affichée (ex. disque 500 Go $\approx$ 465 Gio).





Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+